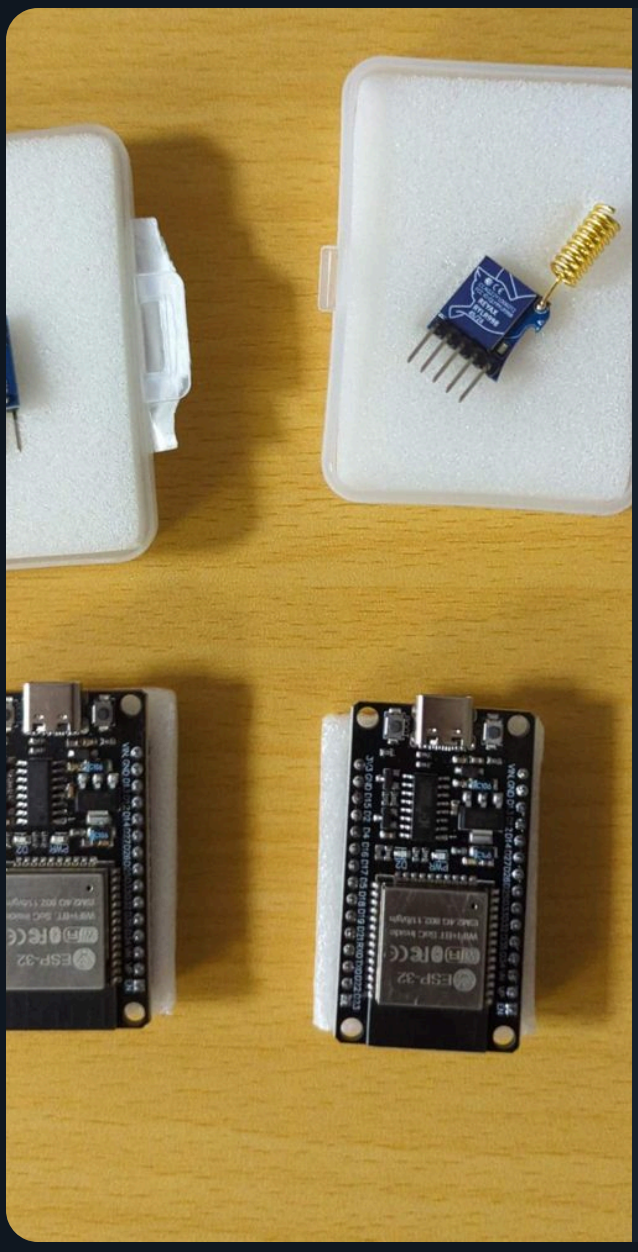
GUÍA DEL PROYECTO · ESP32 + LORA

Radio LoRa híbrida con chat en el celular

Dos estaciones que se comunican a varios kilómetros sin internet ni señal celular: texto, SOS, fotos y notas de voz, y una página web para seguirlo todo desde cualquier lugar.



Lo único que necesitas

- **2 × ESP32 DevKit** (30 pines, USB-C)
- **2 × REYAX RYLR998** (LoRa 915 MHz) con su antena
- Cables dupont hembra-hembra
- Celulares (los que ya tienes)
- Opcional: power bank y una caja

Qué hace

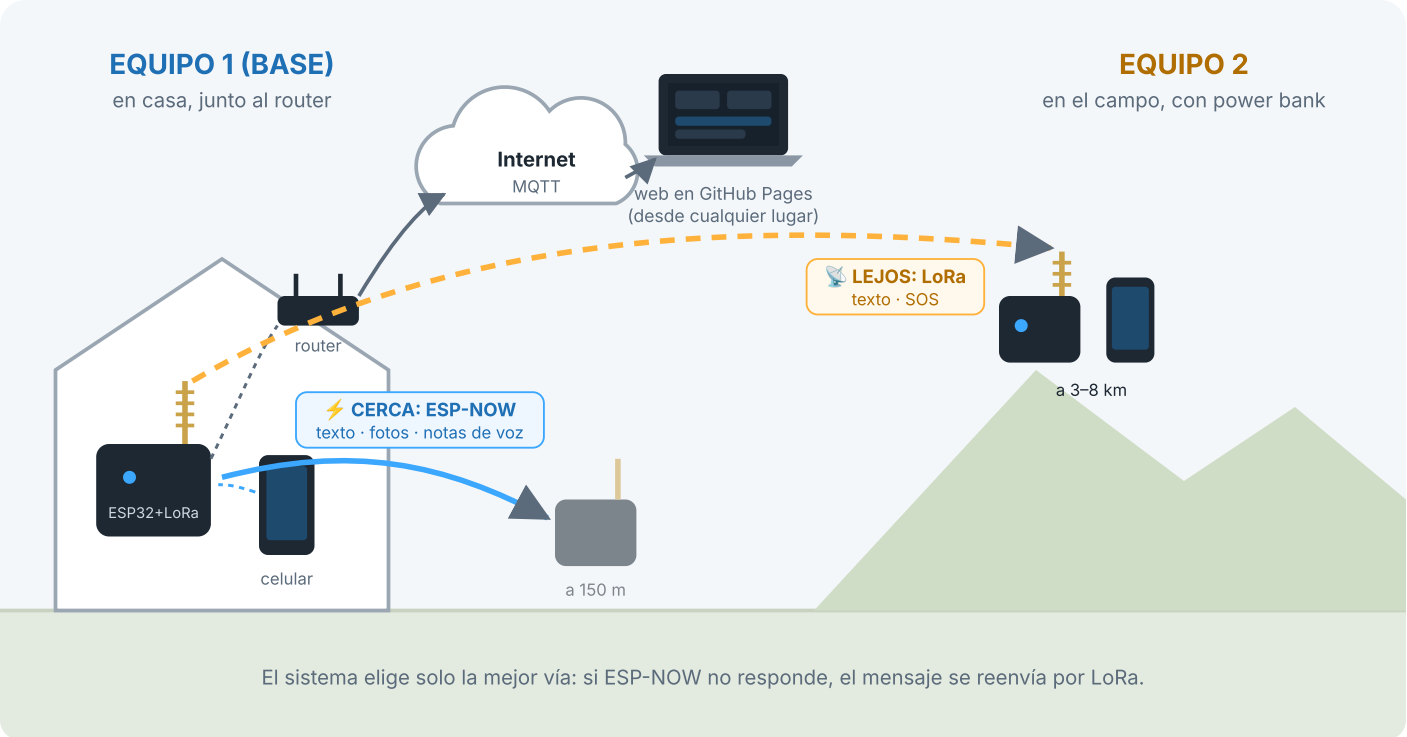
- 💬 Chat con confirmación de entrega
- 🚨 SOS desde el celular o el botón BOOT
- 📷 Fotos y 🎤 notas de voz (de cerca)
- 📡 Texto por LoRa a varios km
- 🌐 Página web en GitHub

Contenido

1. Qué vamos a construir
2. Materiales y prueba de los módulos
3. Conexiones
4. Así queda el montaje
5. Cómo elige la ruta
6. Instalar el software
7. Configurar las dos placas
8. Usarlo desde el celular
9. Código del ESP32 explicado
10. Código de la página del chat
11. Página web en GitHub
12. Pruebas de alcance
13. Solución de problemas
14. Límites y próximos pasos
15. **Proceso del sistema**

1 Qué vamos a construir

Dos estaciones idénticas. Cada una crea su propia red WiFi; te conectas con el celular y abres un chat. Los mensajes viajan entre estaciones por dos radios a la vez:



	⚡ ESP-NOW (cerca)	📡 LoRa (lejos)
Qué es	Radio WiFi del ESP32, sin router	Módulo RYLR998, 915 MHz
Alcance típico	100–200 m	1–3 km en ciudad; 5–10 km o más con línea de vista
Velocidad	Rápida (decenas de KB/s)	Muy lenta (~1 mensaje corto por segundo)
Lleva	Texto, SOS, fotos, notas de voz	Texto y SOS

2 Materiales y prueba de los módulos

Pieza	Cant.	Uso
ESP32 DevKit 30 pines	2	Cerebro, WiFi para el celular y radio ESP-NOW
REYAX RYLR998 + antena	2	Radio LoRa de largo alcance
Cables dupont hembra-hembra	8	4 por estación
Power bank USB	1–2	Para la estación que va al campo (opcional)
Caja plástica	2	Protección (opcional)

Comprueba que los dos módulos sean RYLR998. Ambos deben decir “REYAX RYLR998” impreso. Un HC-12 o un módulo LoRa de 433 MHz **no** se comunica con un RYLR998. La forma más rápida de saberlo: conecta cada módulo y abre el Monitor Serie. Si aparece `LoRa listo`, el módulo respondió a los comandos del RYLR998. Si dice `El RYLR998 no responde`, revisa los cables o el modelo.

3 Conexiones

Conexiones: solo 4 cables por estación (iguales en las dos)



Botón BOOT = SOS

Mantenerlo pulsado 2 segundos envía una emergencia a la otra estación (sin usar el celular).

LED azul

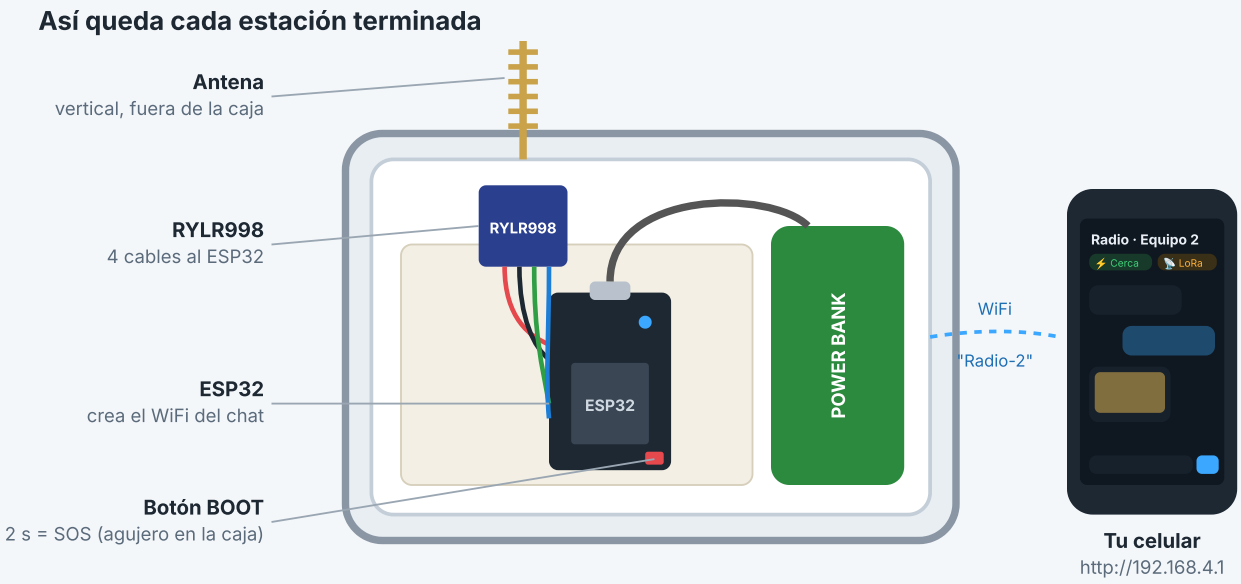
Parpadea al recibir; rápido = SOS.

Importante

- El RYLR998 funciona a 3.3 V: nunca lo conectes a VIN (5 V).
- TX y RX van cruzados.
- No transmitas sin la antena puesta: puede dañarse.

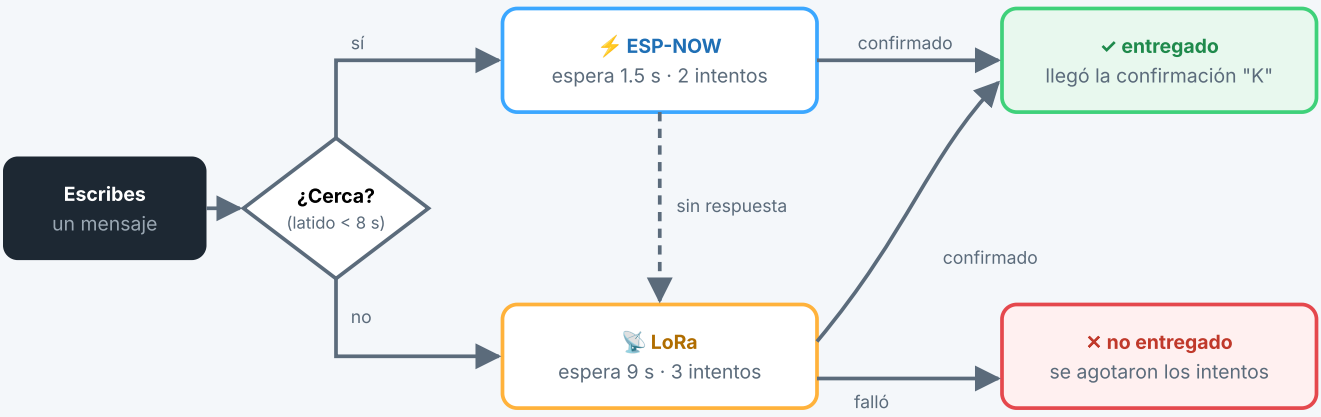
RYLR998	ESP32	Nota
VDD	3V3	Nunca a VIN (5 V)
GND	GND	
TXD	RX2 (GPIO16)	Lo que dice el módulo entra al ESP32
RXD	TX2 (GPIO17)	Lo que dice el ESP32 entra al módulo
NRST	—	Sin conectar

4 Así queda el montaje



- La **antena siempre vertical y fuera de la caja**. Ganar altura es lo que más aumenta el alcance: 2 m más arriba pueden duplicar la distancia.
- Haz un agujero frente al botón BOOT para poder pulsar SOS con un palillo o un botón externo.
- La base se alimenta con un cargador USB cerca del router; la de campo con un power bank (el consumo es de unos 120–150 mA, así que un power bank de 10 000 mAh dura más de un día).

5 Cómo elige la ruta



Fotos y notas de voz NO pasan a LoRa: solo se envían cuando el enlace cercano está activo.

Cada estación envía un “latido” por ESP-NOW cada 2 s. Si la otra lo escuchó hace menos de 8 s, están **cerca** y el mensaje sale por ESP-NOW. Si no, sale directo por LoRa. Todo mensaje debe ser confirmado por el receptor; si no llega la confirmación, se reintenta y, si hace falta, cambia de vía.

6 Instalar el software

1. Instala **Arduino IDE 2** (arduino.cc).
2. *Archivo* → *Preferencias* → "URLs adicionales del gestor de placas": `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. *Herramientas* → *Placa* → *Gestor de placas*: instala **esp32** de Espressif, versión **3.x**.
4. *Herramientas* → *Gestionar bibliotecas*: instala **PubSubClient** (solo hace falta para la base).
5. Abre la carpeta `radio_lora`: deben verse dos pestañas, `radio_lora.ino` y `pagina.h`.
6. Elige la placa **ESP32 Dev Module** y el puerto. Si Windows no la detecta, instala el driver **CH340**.

> El código de esta guía se compiló con el núcleo ESP32 3.3.2 en sus dos versiones (base y remota) sin errores ni advertencias. Ocupa el 77 % de la memoria de programa.

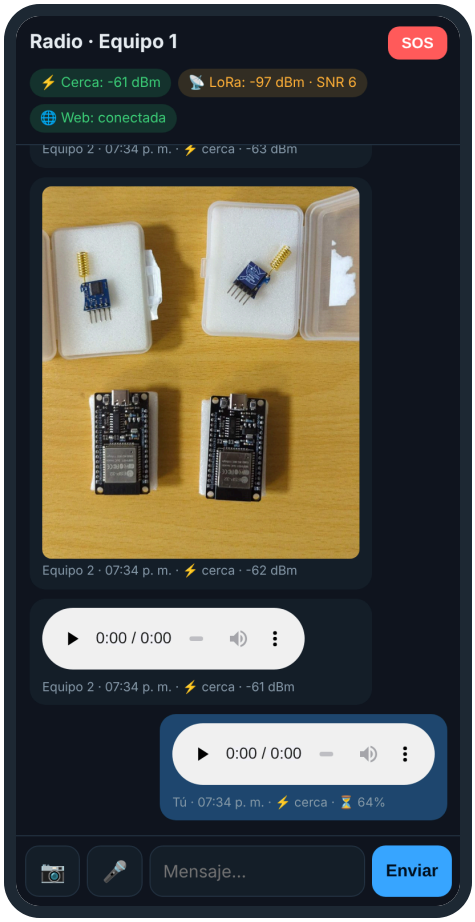
7 Configurar las dos placas

Ajuste	Placa 1 (casa)	Placa 2 (campo)
`EQUIPO`	1	2
`ES_BASE`	1 (con web) o 0	0
`WIFI_SSID / WIFI_PASS`	Tu WiFi	(no importa)
`CANAL_WIFI`	(lo toma del router)	El canal que muestra la placa 1
`TOPIC_PREFIX`	Tu nombre único	(no importa)
`LORA_CLAVE`, `LORA_RED`, `LORA_PARAM`	Iguales en las dos	

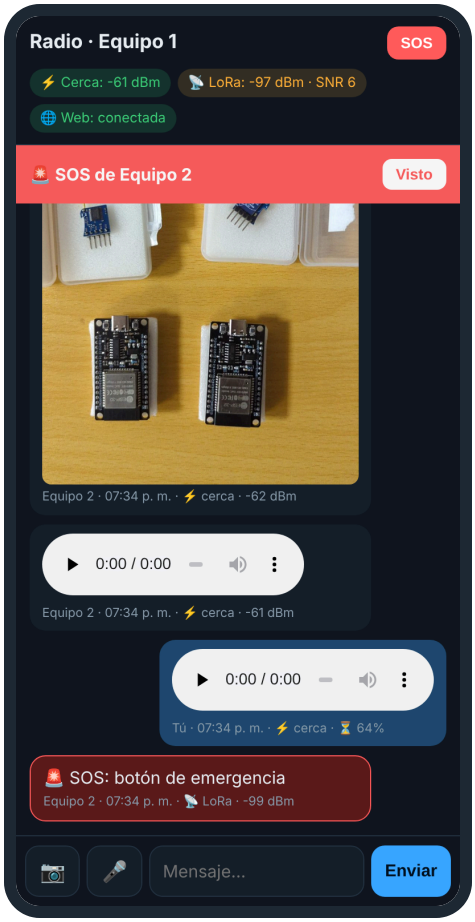
1. Sube el programa a la placa 1 y abre el Monitor Serie a **115200**. Verás algo como `WiFi OK. >>> CANAL 6`.
2. En la placa 2 pon `EQUIPO 2`, `ES_BASE 0` y `CANAL_WIFI 6`, y súbelo.
3. Si no usas base, deja `CANAL_WIFI 1` en las dos.

> Si tu router cambia de canal solo (modo "automático"), la placa 2 dejará de oír por ESP-NOW (LoRa seguirá funcionando). Fija el canal 1, 6 u 11 en la configuración del router.

8 Usarlo desde el celular



Chat normal: los chips muestran la señal de cada enlace.



Al recibir un SOS: franja roja, vibración y LED parpadeando.

1. En el celular, conéctate a la red **Radio-1** (o Radio-2). Clave: `radio1234`.
2. Si pregunta “Esta red no tiene internet, ¿mantener conexión?”, responde **Sí**. Desactiva los datos móviles si el chat no carga.
3. Abre el navegador en `http://192.168.4.1`.
4. Escribe y envía. Verás ⏳ mientras viaja y ✔ **entregado** cuando la otra estación lo confirma.
5. 📷 y 🎤 solo se activan cuando la otra estación está cerca. La nota de voz se graba con la app del celular: que sea corta (10–20 s).

9 Código del ESP32 explicado

Archivo `radio_lora.ino`, **completo**, en sus 13 secciones. Los números de la izquierda son los números de línea.

Cabecera

Comentario que resume el proyecto: cada estación es un ESP32 con un RYLR998; tu celular se conecta al WiFi de la placa y usa el chat. Cerca se usa ESP-NOW; lejos, LoRa; el cambio es automático.

```
/*
  ============================================================================
   RADIO HÍBRIDA ESP32 + LoRa (REYAX RYLR998)
  ============================================================================
   Cada estación = 1 ESP32 + 1 RYLR998. Tu celular se conecta al WiFi que
   crea el ESP32 y abre un chat en http://192.168.4.1

   - CERCA  (ESP-NOW, ~100-200 m): texto, notas de voz y fotos.
   - LEJOS  (LoRa, varios km):     texto y SOS.
   - El cambio entre CERCA y LEJOS es automático.
   - Opcional: la estación BASE se conecta al WiFi de tu casa y publica todo
     en una página web (GitHub Pages) mediante MQTT.

   Arduino IDE: placa "ESP32 Dev Module", núcleo ESP32 3.x
   Librería: "PubSubClient" (solo si ES_BASE = 1)
  ============================================================================
*/
```

1) Configuración

Lo único que normalmente cambias:

- `EQUIPO`: **1** en una placa y **2** en la otra. Es también su dirección LoRa.
- `ES_BASE`: **1** solo en la placa que queda en casa y se conecta a internet. Si no quieres la web, déjalo en 0 en ambas.
- `CLAVE_AP`: contraseña del WiFi que crea cada placa (“Radio-1”, “Radio-2”).
- `CANAL_WIFI`: ESP-NOW exige que las dos placas usen el mismo canal. Si hay base, la base usa el canal de tu router y lo muestra al arrancar; copia ese número en la otra placa.
- **LoRa**: `LORA_RED` (NETWORKID 18), `LORA_BANDA` (915 MHz), `LORA_PARAM` = *9,7,1,12* (factor de dispersión 9, ancho de banda 125 kHz, corrección 4/5, preámbulo 12: la configuración que recomienda REYAX) y `LORA_CLAVE`, una contraseña de 8 dígitos hexadecimales que cifra los mensajes: solo los módulos con la misma clave se entienden. **Cámbiala por una tuya.**
- `LED_PIN` y `BOTON_SOS`: usan el LED azul y el botón BOOT que ya trae la placa. No hay que comprar nada.

```
// ----------------------------------------------------------------------------
// 1) CONFIGURACIÓN
// ----------------------------------------------------------------------------
#define EQUIPO   1          // 1 en una placa, 2 en la otra
#define ES_BASE  0          // 1 = se conecta al WiFi de casa y a la web (solo UNA placa)

// WiFi que crea la placa para tu celular
const char* CLAVE_AP = "radio1234";        // mínimo 8 caracteres
#define CANAL_WIFI 1                       // igual en las dos placas (ver guía)

// Solo para la BASE
const char* WIFI_SSID    = "TU_WIFI";
const char* WIFI_PASS    = "TU_CLAVE";
const char* MQTT_HOST    = "broker.hivemq.com";
```

```
const int   MQTT_PORT    = 1883;
const char* TOPIC_PREFIX = "panarepara-radio-acbdcce2";  // ¡cámbialo! igual en la web

// LoRa (RYLR998): debe ser igual en las dos placas
#define LORA_RX_PIN  16                  // RX2 del ESP32 <- TXD del RYLR998
#define LORA_TX_PIN  17                  // TX2 del ESP32 -> RXD del RYLR998
#define LORA_BAUD    115200
#define LORA_RED     18                  // NETWORKID (0-15 o 18)
#define LORA_BANDA   "915000000"         // Hz (verifica la banda permitida en tu país)
#define LORA_PARAM   "9,7,1,12"          // SF9, 125 kHz, CR 4/5, preámbulo 12
#define LORA_CLAVE   "4F7A21C9"          // cifrado: 8 dígitos hex, igual en ambas

#define LED_PIN   2                      // LED azul de la placa
#define BOTON_SOS 0                       // botón BOOT: mantener 2 s = SOS
```

2) Librerías

`WebServer` y `DNSServer` sirven la página al celular; `LittleFS` usa 1.5 MB de la memoria flash del ESP32 como "disco" para fotos y audios; `vector` son listas que crecen; `pagina.h` es el archivo con la página del chat. `PubSubClient` (MQTT) solo se incluye en la base.

```
// ---------------------------------------------------------------------------
// 2) LIBRERÍAS
// ---------------------------------------------------------------------------
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <WebServer.h>
#include <DNSServer.h>
#include <LittleFS.h>
#include <vector>
#include "pagina.h"                      // la página del chat (HTML)
#if ES_BASE
#include <PubSubClient.h>
#endif
```

3) Constantes y tipos

- `OTRO` : la dirección LoRa de la otra placa (si soy 1, es 2).
- Límites: 40 mensajes en memoria, 180 bytes por texto (LoRa admite 240), archivos de hasta 400 KB y trozos de 236 bytes por paquete ESP-NOW.
- `CERCA_MS` : si en 8 s no llegó nada por ESP-NOW, la otra estación se considera "lejos".
- `Msg` es un mensaje del chat: quién lo escribió, tipo (texto, SOS, audio, foto), estado (enviando, entregado, fallido, recibido), por qué vía viajó y con qué señal llegó.
- `version` aumenta con cada cambio; así la página sabe cuándo hay novedades. `Pendiente` guarda los textos que esperan confirmación.

```
// ---------------------------------------------------------------------------
// 3) CONSTANTES Y TIPOS
// ---------------------------------------------------------------------------
const uint8_t OTRO = (EQUIPO == 1) ? 2 : 1;   // dirección LoRa de la otra placa
const uint8_t BCAST[6] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

#define MAX_MSGS        40       // mensajes guardados en memoria
#define MAX_TEXTO       180      // bytes por mensaje de texto
#define MAX_ARCHIVO     400000   // tamaño máximo de foto / nota de voz
#define CHUNK           236      // bytes de archivo por paquete ESP-NOW
#define CERCA_MS        8000     // sin noticias ESP-NOW en 8 s = lejos
#define LORA_VIVO_MS    75000    // sin noticias LoRa en 75 s = sin enlace

enum Tipo   : uint8_t { TEXTO = 0, SOS = 1, AUDIO = 2, FOTO = 3 };
```

```cpp
enum Estado : uint8_t { ENVIANDO = 0, ENTREGADO = 1, FALLIDO = 2, RECIBIDO = 3 };

struct Msg {
  uint16_t id;        // número del mensaje (lo pone quien lo envía)
  uint8_t  de;        // equipo que lo escribió (0 = página web)
  bool     mio;       // true si salió de esta placa
  uint8_t  tipo;      // TEXTO, SOS, AUDIO, FOTO
  uint8_t  estado;    // ENVIANDO, ENTREGADO...
  char     via;       // 'E' ESP-NOW, 'L' LoRa, '-' aún no
  int16_t  rssi;      // señal con la que llegó
  uint8_t  progreso;  // % de envío de archivos
  String   texto;     // texto, o ruta del archivo
};
std::vector<Msg> msgs;
uint32_t version = 1;          // cambia cada vez que algo cambia (para la web)

// Mensajes de texto esperando confirmación
struct Pendiente {
  uint16_t id; uint8_t tipo; String texto;
  char via; uint8_t intentos; uint32_t t;
};
std::vector<Pendiente> pendientes;
```

4) Variables globales

Objetos del servidor web, el DNS y el puerto serie del LoRa (`Serial2`). `ultimoEspNow` y `ultimoLora` guardan cuándo se supo de la otra estación por cada vía, y de ahí salen `cerca()` y `loraVivo()` . `colaEsp` es una cola de FreeRTOS donde se guardan los paquetes ESP-NOW que llegan; `colaLora` guarda los envíos LoRa pendientes.

```cpp
// ---------------------------------------------------------------------------
// 4) VARIABLES GLOBALES
// ---------------------------------------------------------------------------
WebServer server(80);
DNSServer dns;
HardwareSerial& lora = Serial2;

uint16_t siguienteId = 1;
uint32_t ultimoEspNow = 0, ultimoLora = 0;
int espRssi = 0, loraRssi = 0, loraSnr = 0;
bool loraOk = false;
uint8_t sosDe = 0;             // equipo que mandó SOS sin atender (0 = ninguno)
uint32_t sosHasta = 0;

// Cola de recepción ESP-NOW (el callback solo copia, el loop procesa)
struct RxEsp { uint8_t d[250]; int len; int rssi; };
QueueHandle_t colaEsp;

// LoRa: cola de envíos (el módulo atiende un comando a la vez)
std::vector<String> colaLora;
bool loraOcupado = false;
uint32_t loraDesde = 0;
String lineaLora;

#if ES_BASE
WiFiClient net;
PubSubClient mqtt(net);
String T(const char* s) { return String(TOPIC_PREFIX) + "/" + s; }
#endif

bool cerca() { return ultimoEspNow && millis() - ultimoEspNow < CERCA_MS; }
bool loraVivo() { return ultimoLora && millis() - ultimoLora < LORA_VIVO_MS; }
```

5) Lista de mensajes

`buscar()` encuentra un mensaje por equipo + número. `agregar()` lo añade y, si ya hay 40, borra el más viejo (y su foto o audio de la memoria). `parpadear()` enciende el LED y `limpiar()` quita saltos de línea (el RYLR998 no los acepta dentro de un mensaje) y recorta textos largos.

```
// ---------------------------------------------------------------------------
// 5) LISTA DE MENSAJES
// ---------------------------------------------------------------------------
Msg* buscar(uint8_t de, uint16_t id, bool mio) {
  for (auto& m : msgs)
    if (m.id == id && m.de == de && m.mio == mio) return &m;
  return nullptr;
}

Msg& agregar(const Msg& m) {
  if (msgs.size() >= MAX_MSGS) {                // lleno: borrar el más viejo
    Msg& v = msgs.front();
    if ((v.tipo == AUDIO || v.tipo == FOTO) && v.texto.length())
      LittleFS.remove(v.texto);
    msgs.erase(msgs.begin());
  }
  msgs.push_back(m);
  version++;
  return msgs.back();
}

void parpadear(int veces) {
  for (int i = 0; i < veces; i++) {
    digitalWrite(LED_PIN, HIGH); delay(40);
    digitalWrite(LED_PIN, LOW);  delay(40);
  }
}

String limpiar(String s) {                      // sin saltos de línea
  s.replace("\r", " "); s.replace("\n", " ");
  while (s.length() > MAX_TEXTO) s.remove(s.length() - 1);
  return s;
}
```

6) LoRa: RYLR998 por comandos AT

El RYLR998 se maneja enviándole texto por el puerto serie.

- `loraComando()` envía un comando y espera `+OK`. Solo se usa al arrancar.
- `loraIniciar()` comprueba que el módulo responde a `AT` y lo configura: dirección, red, frecuencia, parámetros de radio, clave de cifrado y potencia máxima (22 dBm). La clave (`AT+CPIN`) se borra al reiniciar el módulo, por eso se envía siempre.
- `loraEnviar()` arma `AT+SEND=2,13,M,1,857,0,Hola`: dirección destino, largo en bytes y datos.
- `loraLoop()` manda un comando a la vez (el módulo está ocupado mientras transmite) y lee lo que llega. Un mensaje recibido se ve así: `+RCV=1,13,M,1,857,0,Hola,-97,6` → dirección, largo, datos, **RSSI** (fuerza en dBm) y **SNR** (calidad: con SF9 aún se recibe hasta unos −12 dB).

```
// ---------------------------------------------------------------------------
// 6) LoRa (RYLR998 con comandos AT)
// ---------------------------------------------------------------------------
// Envía un comando y espera "+OK" (solo se usa al arrancar)
bool loraComando(const String& cmd, uint32_t espera = 1500) {
  while (lora.available()) lora.read();
  lora.print(cmd + "\r\n");
  uint32_t t0 = millis();
  String l;
  while (millis() - t0 < espera) {
```

```
    while (lora.available()) {
      char c = lora.read();
      if (c == '\n') {
        l.trim();
        if (l.startsWith("+OK")) return true;
        if (l.startsWith("+ERR")) { Serial.printf("LoRa %s -> %s\n", cmd.c_str(), l.c_str()); return false;
}
        l = "";
      } else l += c;
    }
  }
  return false;
}

void loraIniciar() {
  lora.begin(LORA_BAUD, SERIAL_8N1, LORA_RX_PIN, LORA_TX_PIN);
  delay(200);
  loraOk = loraComando("AT");
  if (!loraOk) {
    Serial.println("!! El RYLR998 no responde: revisa TXD->16, RXD->17, 3V3 y GND");
    return;
  }
  loraComando("AT+ADDRESS=" + String(EQUIPO));
  loraComando("AT+NETWORKID=" + String(LORA_RED));
  loraComando("AT+BAND=" + String(LORA_BANDA));
  loraComando("AT+PARAMETER=" + String(LORA_PARAM));
  loraComando("AT+CPIN=" + String(LORA_CLAVE));
  loraComando("AT+CRFOP=22");                      // potencia máxima (22 dBm)
  Serial.println("LoRa listo");
}

// Encola un envío LoRa a la otra placa
void loraEnviar(const String& datos) {
  if (!loraOk) return;
  colaLora.push_back("AT+SEND=" + String(OTRO) + "," + String(datos.length()) + "," + datos);
}

void procesarTramaTexto(const String& s, char via, int rssi);

// Atiende el módulo: manda lo pendiente y lee lo que llega
void loraLoop() {
  if (!loraOk) return;
  if (!loraOcupado && !colaLora.empty()) {
    lora.print(colaLora.front() + "\r\n");
    colaLora.erase(colaLora.begin());
    loraOcupado = true;
    loraDesde = millis();
  }
  if (loraOcupado && millis() - loraDesde > 5000) loraOcupado = false;  // por si acaso

  while (lora.available()) {
    char c = lora.read();
    if (c != '\n') { if (lineaLora.length() < 300) lineaLora += c; continue; }
    lineaLora.trim();
    if (lineaLora.startsWith("+OK") || lineaLora.startsWith("+ERR")) {
      loraOcupado = false;
    } else if (lineaLora.startsWith("+RCV=")) {
      // Formato: +RCV=<dirección>,<largo>,<datos>,<RSSI>,<SNR>
      int p1 = lineaLora.indexOf(',');
      int p2 = lineaLora.indexOf(',', p1 + 1);
      int largo = lineaLora.substring(p1 + 1, p2).toInt();
      int ini = p2 + 1;
      while (ini < (int)lineaLora.length() && lineaLora[ini] == ' ') ini++;  // por si trae espacios
      String datos = lineaLora.substring(ini, ini + largo);
      String resto = lineaLora.substring(ini + largo + 1);
      int p3 = resto.indexOf(',');
      loraRssi = resto.substring(0, p3).toInt();
```

```
      loraSnr  = resto.substring(p3 + 1).toInt();
      ultimoLora = millis();
      version++;
      procesarTramaTexto(datos, 'L', loraRssi);
    }
    lineaLora = "";
  }
}
```

7) ESP-NOW

El callback `onEspNow()` se ejecuta en la tarea interna del WiFi, así que solo copia el paquete a la cola y termina rápido; el `loop` lo procesa con calma. Los envíos van a la dirección de difusión FF:FF:FF:FF:FF:FF, así no hace falta conocer la MAC de la otra placa.

```
// ---------------------------------------------------------------------------
// 7) ESP-NOW (radio directa entre ESP32)
// ---------------------------------------------------------------------------
void onEspNow(const esp_now_recv_info_t* info, const uint8_t* data, int len) {
  if (len <= 0 || len > 250) return;
  RxEsp r;
  memcpy(r.d, data, len);
  r.len = len;
  r.rssi = info->rx_ctrl ? info->rx_ctrl->rssi : 0;
  xQueueSend(colaEsp, &r, 0);                    // el loop lo procesa
}

void espEnviar(const uint8_t* d, size_t n) { esp_now_send(BCAST, d, n); }
void espEnviarTexto(const String& s) { espEnviar((const uint8_t*)s.c_str(), s.length()); }
```

8) Protocolo de texto

El mismo formato sirve por ESP-NOW y por LoRa, y es texto legible:

- `M,1,857,0,Hola` → mensaje del equipo 1, número 857, tipo 0 (texto; 1 = SOS).
- `K,2,857` → el equipo 2 confirma que recibió el 857.
- `P,1` → latido: “sigo aquí”.

`procesarTramaTexto()` interpreta lo que llega: confirma **siempre por la misma vía** (aunque sea un mensaje repetido, por si se perdió la confirmación anterior) y descarta duplicados. `enviarMensaje()` elige la vía y deja el mensaje pendiente. `revisarPendientes()` hace los reintentos del diagrama de la sección 5.

```
// ---------------------------------------------------------------------------
// 8) PROTOCOLO DE TEXTO (igual por ESP-NOW y por LoRa)
//    M,<de>,<id>,<tipo>,<texto>   mensaje
//    K,<de>,<id>                  confirmación ("lo recibí")
//    P,<de>                       "estoy aquí" (latido)
// ---------------------------------------------------------------------------
void enviarPorVia(char via, const String& trama) {
  if (via == 'E') espEnviarTexto(trama);
  else loraEnviar(trama);
}

void publicarMsg(const Msg& m);   // (MQTT, más abajo)

void procesarTramaTexto(const String& s, char via, int rssi) {
  if (s.length() < 3 || s[1] != ',') return;
  char tipoTrama = s[0];
  int c1 = s.indexOf(',', 2);
  uint8_t de = s.substring(2, c1 < 0 ? s.length() : c1).toInt();
  if (de == EQUIPO) return;                      // eco de mí mismo

  if (tipoTrama == 'P') return;                  // latido: ya se anotó la señal

```

```
  int c2 = s.indexOf(',', c1 + 1);
  uint16_t id = s.substring(c1 + 1, c2 < 0 ? s.length() : c2).toInt();

  if (tipoTrama == 'K') {                          // confirmación de algo mío
    for (size_t i = 0; i < pendientes.size(); i++)
      if (pendientes[i].id == id) { pendientes.erase(pendientes.begin() + i); break; }
    Msg* m = buscar(EQUIPO, id, true);
    if (!m) m = buscar(0, id, true);
    if (m && m->estado != ENTREGADO) { m->estado = ENTREGADO; m->via = via; version++; }
    return;
  }

  if (tipoTrama == 'M') {
    int c3 = s.indexOf(',', c2 + 1);
    if (c3 < 0) return;
    uint8_t tipo = s.substring(c2 + 1, c3).toInt();
    String texto = s.substring(c3 + 1);
    // Confirmar SIEMPRE por la misma vía (aunque sea repetido)
    enviarPorVia(via, "K," + String(EQUIPO) + "," + String(id));
    if (buscar(de, id, false)) return;             // repetido: ya lo tengo
    Msg m{id, de, false, tipo, RECIBIDO, via, (int16_t)rssi, 100, texto};
    agregar(m);
    publicarMsg(m);
    Serial.printf("[%s %s] %s\n", via == 'E' ? "CERCA" : "LoRa",
                  tipo == SOS ? "SOS" : "MSG", texto.c_str());
    if (tipo == SOS) { sosDe = de; sosHasta = millis() + 120000; }
    parpadear(3);
  }
}

// Envía un mensaje de texto o SOS, eligiendo la mejor vía
void enviarMensaje(uint8_t tipo, String texto, uint8_t de = EQUIPO) {
  texto = limpiar(texto);
  if (!texto.length()) return;
  uint16_t id = siguienteId++;
  char via = cerca() ? 'E' : 'L';
  Msg m{id, de, true, tipo, ENVIANDO, via, 0, 0, texto};
  agregar(m);
  publicarMsg(m);
  String trama = "M," + String(EQUIPO) + "," + String(id) + "," + String(tipo) + "," + texto;
  enviarPorVia(via, trama);
  pendientes.push_back({id, tipo, trama, via, 1, millis()});
  Serial.printf("[ENVIADO por %s] %s\n", via == 'E' ? "CERCA" : "LoRa", texto.c_str());
}

// Reintentos: por ESP-NOW espera 1.5 s (2 intentos); si no hay respuesta,
// pasa a LoRa y espera 9 s (3 intentos). Si nada funciona: "no entregado".
void revisarPendientes() {
  for (size_t i = 0; i < pendientes.size(); i++) {
    Pendiente& p = pendientes[i];
    uint32_t espera = (p.via == 'E') ? 1500 : 9000;
    if (millis() - p.t < espera) continue;

    bool reintentar = true;
    if (p.via == 'E') {
      if (p.intentos < 2 && cerca()) p.intentos++;             // otra vez cerca
      else if (loraOk) { p.via = 'L'; p.intentos = 1; }        // saltar a LoRa
      else reintentar = false;
    } else {
      if (p.intentos < 3) p.intentos++;
      else reintentar = false;
    }

    Msg* m = buscar(EQUIPO, p.id, true);
    if (!m) m = buscar(0, p.id, true);
    if (!reintentar) {
      if (m) { m->estado = FALLIDO; version++; }
```

```
      pendientes.erase(pendientes.begin() + i);
      i--;
      continue;
    }
    enviarPorVia(p.via, p.texto);
    p.t = millis();
    if (m && m->via != p.via) { m->via = p.via; version++; }
  }
}
```

9) Archivos: fotos y notas de voz

Un archivo de 100 KB necesita unos 430 paquetes, así que se envía en trozos con confirmación (método “parar y esperar”):

- **f** (inicio): número, tipo, tamaño y extensión.
- **d** (datos): número de trozo + 236 bytes.
- **a** (confirmación): “ya tengo todo hasta el trozo n−1, mándame el n”.

Si no llega confirmación en 80 ms se repite el trozo; tras 60 fallos seguidos se marca “no entregado”. El receptor escribe directo en la memoria flash, así que no gasta RAM. A unos 20–40 KB/s, una foto tarda pocos segundos.

```
// ---------------------------------------------------------------------------
// 9) ARCHIVOS (fotos y notas de voz) — solo por ESP-NOW, en trozos
//    f <id:2> <tipo:1> <tamaño:4> <ext>   inicio
//    d <id:2> <n:2> <datos>               trozo número n
//    a <id:2> <siguiente:2>               "ya tengo hasta n-1, mándame n"
// ---------------------------------------------------------------------------
struct Envio {
  bool activo = false; uint16_t id; uint8_t tipo; File f; uint32_t tam;
  uint16_t total, n; uint32_t t; uint16_t fallos; String ext;
} tx;
struct Recibo {
  bool activo = false; uint16_t id; uint8_t de; uint8_t tipo; File f;
  uint32_t tam, escritos; uint16_t siguiente; String ruta; uint32_t t; int rssi;
} rx;

void txMandarTrozo() {
  uint8_t b[250];
  if (tx.n == 0xFFFF) {                          // todavía no aceptó el inicio
    b[0] = 'f'; memcpy(b + 1, &tx.id, 2); b[3] = tx.tipo; memcpy(b + 4, &tx.tam, 4);
    size_t e = tx.ext.length();
    memcpy(b + 8, tx.ext.c_str(), e);
    espEnviar(b, 8 + e);
  } else {
    b[0] = 'd'; memcpy(b + 1, &tx.id, 2); memcpy(b + 3, &tx.n, 2);
    tx.f.seek((uint32_t)tx.n * CHUNK);
    size_t r = tx.f.read(b + 5, CHUNK);
    espEnviar(b, 5 + r);
  }
  tx.t = millis();
}

bool empezarEnvioArchivo(const String& ruta, uint8_t tipo, const String& ext) {
  if (tx.activo || !cerca()) return false;
  tx.f = LittleFS.open(ruta, "r");
  if (!tx.f) return false;
  tx.activo = true; tx.id = siguienteId++; tx.tipo = tipo; tx.tam = tx.f.size();
  tx.total = (tx.tam + CHUNK - 1) / CHUNK; tx.n = 0xFFFF; tx.fallos = 0; tx.ext = ext;
  Msg m{tx.id, EQUIPO, true, tipo, ENVIANDO, 'E', 0, 0, ruta};
  agregar(m);
  publicarMsg(m);
  txMandarTrozo();
  return true;
}
```

```
409
410 void txTerminar(bool ok) {
411   tx.f.close();
412   tx.activo = false;
413   Msg* m = buscar(EQUIPO, tx.id, true);
414   if (m) { m->estado = ok ? ENTREGADO : FALLIDO; m->progreso = ok ? 100 : m->progreso; version++; }
415 }
416
417 void txLoop() {
418   if (!tx.activo) return;
419   if (millis() - tx.t > 80) {                       // sin respuesta: repetir
420     if (++tx.fallos > 60) { txTerminar(false); return; }
421     txMandarTrozo();
422   }
423 }
424
425 void procesarArchivo(const uint8_t* d, int len, int rssi) {
426   if (len < 3) return;
427   uint16_t id; memcpy(&id, d + 1, 2);
428   if (d[0] == 'a' && tx.activo && id == tx.id && len >= 5) {   // soy el que envía
429     uint16_t sig; memcpy(&sig, d + 3, 2);
430     tx.fallos = 0;
431     if (sig >= tx.total) { txTerminar(true); return; }
432     if (tx.n == 0xFFFF || sig > tx.n) {
433       tx.n = sig;
434       Msg* m = buscar(EQUIPO, tx.id, true);
435       uint8_t p = (uint32_t)sig * 100 / tx.total;
436       if (m && m->progreso != p) { m->progreso = p; version++; }
437     }
438     txMandarTrozo();
439     return;
440   }
441   uint8_t r[5] = {'a'};
442   if (d[0] == 'f' && len >= 8) {                    // soy el que recibe: inicio
443     if (!(rx.activo && rx.id == id)) {
444       if (rx.activo) { rx.f.close(); LittleFS.remove(rx.ruta); }
445       uint32_t tam; memcpy(&tam, d + 4, 4);
446       String ext((const char*)(d + 8), len - 8);
447       rx.activo = true; rx.id = id; rx.tipo = d[3]; rx.tam = tam; rx.escritos = 0;
448       rx.siguiente = 0; rx.rssi = rssi;
449       rx.ruta = "/m/r" + String(id) + "_" + String(millis() % 100000) + "." + ext;
450       rx.f = LittleFS.open(rx.ruta, "w");
451     }
452     rx.t = millis();
453     memcpy(r + 1, &id, 2); memcpy(r + 3, &rx.siguiente, 2);
454     espEnviar(r, 5);
455     return;
456   }
457   // (si ya terminó y el emisor repite el último trozo, solo se vuelve a confirmar)
458   if (d[0] == 'd' && len >= 5 && id == rx.id && (rx.activo || rx.siguiente > 0)) {
459     uint16_t n; memcpy(&n, d + 3, 2);
460     if (rx.activo && n == rx.siguiente) {
461       rx.f.write(d + 5, len - 5);
462       rx.escritos += len - 5;
463       rx.siguiente++;
464     }
465     rx.t = millis();
466     memcpy(r + 1, &id, 2); memcpy(r + 3, &rx.siguiente, 2);
467     espEnviar(r, 5);
468     if (rx.activo && rx.escritos >= rx.tam) {      // ¡completo!
469       rx.f.close();
470       rx.activo = false;
471       if (!buscar(OTRO, id, false)) {
472         Msg m{id, OTRO, false, rx.tipo, RECIBIDO, 'E', (int16_t)rx.rssi, 100, rx.ruta};
473         agregar(m);
474         publicarMsg(m);
475         parpadear(3);
```

```
        Serial.printf("[CERCA] %s recibida (%u bytes)\n", rx.tipo == FOTO ? "Foto" : "Nota de voz", rx.tam);
      }
    }
  }
}

void rxLoop() {                                   // recepción abandonada
  if (rx.activo && millis() - rx.t > 8000) {
    rx.f.close(); LittleFS.remove(rx.ruta); rx.activo = false;
  }
}
```

10) Página web del celular

El ESP32 es un pequeño servidor web:

- `/` entrega la página del chat; `/api` devuelve en JSON el estado de los enlaces y los mensajes (la página lo pide cada segundo).
- `/enviar` recibe un texto o un SOS; `/sosvisto` apaga la alarma.
- `/subir` recibe una foto o audio y lo guarda en la flash mientras llega (no cabe en RAM); luego inicia el envío por radio. Si la otra estación está lejos, responde con un aviso.
- `/f/...` entrega los archivos con el tipo correcto para que el celular los muestre o reproduzca.
- Las rutas `/generate_204` y `/hotspot-detect.html` son las que usan Android y iPhone para comprobar si hay internet; responderlas evita que el celular se desconecte por “red sin internet”. El DNS responde a cualquier dirección con la IP de la placa.

```
// ---------------------------------------------------------------------------
// 10) PÁGINA WEB DEL CELULAR
// ---------------------------------------------------------------------------
String jsonEsc(const String& s) {
  String o; o.reserve(s.length() + 8);
  for (size_t i = 0; i < s.length(); i++) {
    char c = s[i];
    if (c == '"' || c == '\\') { o += '\\'; o += c; }
    else if ((uint8_t)c < 0x20) o += ' ';
    else o += c;
  }
  return o;
}

void apiEstado() {
  String j; j.reserve(4096);
  j = "{\"v\":" + String(version) + ",\"yo\":" + String(EQUIPO) + ",\"otro\":" + String(OTRO);
  j += ",\"cerca\":{\"on\":" + String(cerca() ? "true" : "false") + ",\"rssi\":" + String(espRssi) + "}";
  j += ",\"lora\":{\"ok\":" + String(loraOk ? "true" : "false") + ",\"on\":" + String(loraVivo() ? "true" :
"false") +
       ",\"rssi\":" + String(loraRssi) + ",\"snr\":" + String(loraSnr) + "}";
  j += ",\"ocupado\":" + String(tx.activo ? "true" : "false");
  j += ",\"sos\":" + String(sosDe);
#if ES_BASE
  j += ",\"web\":" + String(mqtt.connected() ? "true" : "false");
#endif
  j += ",\"msgs\":[";
  for (size_t i = 0; i < msgs.size(); i++) {
    const Msg& m = msgs[i];
    if (i) j += ",";
    j += "{\"id\":" + String(m.id) + ",\"de\":" + String(m.de) + ",\"mio\":" + (m.mio ? "true" : "false") +
         ",\"t\":" + String(m.tipo) + ",\"e\":" + String(m.estado) + ",\"via\":\"" + String(m.via) +
         "\",\"rssi\":" + String(m.rssi) + ",\"p\":" + String(m.progreso) +
         ",\"x\":\"" + jsonEsc((m.tipo == AUDIO || m.tipo == FOTO) ? String("/f") + m.texto : m.texto) +
"\"}";
  }
  j += "]}";
```

```
  server.sendHeader("Cache-Control", "no-store");
  server.send(200, "application/json", j);
}

void apiEnviar() {
  String t = server.arg("t");
  uint8_t tipo = server.arg("sos") == "1" ? SOS : TEXTO;
  if (tipo == SOS && !t.length()) t = "SOS: necesito ayuda";
  enviarMensaje(tipo, t);
  server.send(200, "text/plain", "ok");
}

void apiSosVisto() { sosDe = 0; version++; server.send(200, "text/plain", "ok"); }

// Subida de foto / audio desde el celular
File subida; String subidaRuta; bool subidaMal = false;
void apiSubidaDatos() {
  HTTPUpload& u = server.upload();
  if (u.status == UPLOAD_FILE_START) {
    String ext = server.arg("ext");
    if (ext.length() < 2 || ext.length() > 5) ext = "bin";
    subidaRuta = "/m/t" + String(millis() % 100000) + "." + ext;
    subida = LittleFS.open(subidaRuta, "w");
    subidaMal = !subida;
  } else if (u.status == UPLOAD_FILE_WRITE) {
    if (!subidaMal) {
      if (subida.size() + u.currentSize > MAX_ARCHIVO) subidaMal = true;
      else subida.write(u.buf, u.currentSize);
    }
  } else if (u.status == UPLOAD_FILE_END || u.status == UPLOAD_FILE_ABORTED) {
    if (subida) subida.close();
    if (u.status == UPLOAD_FILE_ABORTED) subidaMal = true;
  }
}
void apiSubidaFin() {
  uint8_t tipo = server.arg("tipo") == "foto" ? FOTO : AUDIO;
  String ext = server.arg("ext");
  if (subidaMal) { LittleFS.remove(subidaRuta); server.send(413, "text/plain", "Archivo muy grande (máx. 400
KB)"); return; }
  if (!empezarEnvioArchivo(subidaRuta, tipo, ext)) {
    LittleFS.remove(subidaRuta);
    server.send(409, "text/plain", "La otra estación está lejos u ocupada: fotos y audios solo viajan CERC
A");
    return;
  }
  server.send(200, "text/plain", "ok");
}

void apiArchivo() {
  String ruta = server.uri().substring(2);         // quita "/f"
  if (!ruta.startsWith("/m/") || !LittleFS.exists(ruta)) { server.send(404, "text/plain", "no"); return; }
  String ct = "application/octet-stream";
  if (ruta.endsWith(".jpg")) ct = "image/jpeg";
  else if (ruta.endsWith(".m4a") || ruta.endsWith(".mp4")) ct = "audio/mp4";
  else if (ruta.endsWith(".aac")) ct = "audio/aac";
  else if (ruta.endsWith(".amr")) ct = "audio/amr";
  else if (ruta.endsWith(".3gp")) ct = "audio/3gpp";
  else if (ruta.endsWith(".ogg") || ruta.endsWith(".opus")) ct = "audio/ogg";
  else if (ruta.endsWith(".webm")) ct = "audio/webm";
  else if (ruta.endsWith(".mp3")) ct = "audio/mpeg";
  else if (ruta.endsWith(".wav")) ct = "audio/wav";
  File f = LittleFS.open(ruta, "r");
  server.streamFile(f, ct);
  f.close();
}

void iniciarWeb() {
```

```
  server.on("/", []() { server.send_P(200, "text/html", PAGINA); });
  server.on("/api", apiEstado);
  server.on("/enviar", HTTP_POST, apiEnviar);
  server.on("/sosvisto", HTTP_POST, apiSosVisto);
  server.on("/subir", HTTP_POST, apiSubidaFin, apiSubidaDatos);
  // Respuestas para que el celular no crea que "no hay internet" y se desconecte
  server.on("/generate_204", []() { server.send(204); });
  server.on("/hotspot-detect.html", []() { server.send(200, "text/html", "Success"); });
  server.onNotFound([]() {
    if (server.uri().startsWith("/f/")) { apiArchivo(); return; }
    server.sendHeader("Location", "http://192.168.4.1/");
    server.send(302);
  });
  server.begin();
  dns.start(53, "*", WiFi.softAPIP());            // cualquier dirección -> esta placa
}
```

11) MQTT (solo la base)

Cuando escribes en la web de GitHub, el mensaje llega a `onMqtt()` y se envía por radio como “Web” con el símbolo 🌐. `mqttLoop()` reconecta cada 5 s si se cae, publica cada 10 s el estado de los dos enlaces y deja un “testamento” para que la web sepa si la base se apaga. `publicarMsg()` sube cada mensaje (las fotos y audios aparecen como texto, porque no pasan por internet).

```
// ---------------------------------------------------------------------------
// 11) MQTT (solo BASE): puente con la página de GitHub
// ---------------------------------------------------------------------------
#if ES_BASE
void onMqtt(char* topic, byte* payload, unsigned int len) {
  String s; for (unsigned i = 0; i < len; i++) s += (char)payload[i];
  bool sos = String(topic).endsWith("/web/sos");
  if (!s.length()) s = "SOS";
  enviarMensaje(sos ? SOS : TEXTO, "🌐 " + s, 0);   // 🌐 = escrito en la web
}

void mqttLoop() {
  if (WiFi.status() != WL_CONNECTED) return;
  if (!mqtt.connected()) {
    static uint32_t intento = 0;
    if (millis() - intento < 5000) return;
    intento = millis();
    String id = "radio-" + String((uint32_t)ESP.getEfuseMac(), HEX);
    if (!mqtt.connect(id.c_str(), T("estado").c_str(), 0, true, "{\"base\":false}")) return;
    mqtt.subscribe(T("web/msg").c_str());
    mqtt.subscribe(T("web/sos").c_str());
    Serial.println("MQTT conectado");
  }
  mqtt.loop();
  static uint32_t ultimo = 0;
  if (millis() - ultimo > 10000) {
    ultimo = millis();
    String j = "{\"base\":true,\"cerca\":{\"on\":" + String(cerca() ? "true" : "false") + ",\"rssi\":" + Str
ing(espRssi) +
               "},\"lora\":{\"on\":" + String(loraVivo() ? "true" : "false") + ",\"rssi\":" + String(loraRss
i) +
               ",\"snr\":" + String(loraSnr) + "}}";
    mqtt.publish(T("estado").c_str(), j.c_str(), true);
  }
}
#endif

void publicarMsg(const Msg& m) {
#if ES_BASE
  if (!mqtt.connected()) return;
  String x = (m.tipo == AUDIO) ? "🎤 Nota de voz" : (m.tipo == FOTO) ? "📷 Foto" : m.texto;
```

```
  String j = "{\"de\":" + String(m.de) + ",\"id\":" + String(m.id) + ",\"t\":" + String(m.tipo) +
             ",\"via\":\"" + String(m.via) + "\",\"x\":\"" + jsonEsc(x) + "\"}";
  mqtt.publish(T("radio/msg").c_str(), j.c_str());
#endif
}
```

12) setup()

Al encender:

1. Borra los archivos viejos de la memoria.
2. Configura el RYLR998.
3. Si es la base, primero se conecta a tu WiFi y toma su canal. El ESP32 solo tiene **una radio WiFi**: su red propia, la conexión a casa y ESP-NOW deben usar el mismo canal.
4. Crea la red “Radio-1” o “Radio-2” para el celular.
5. Inicia ESP-NOW y el servidor web.

```
// ---------------------------------------------------------------------------
// 12) SETUP
// ---------------------------------------------------------------------------
void setup() {
  Serial.begin(115200);
  delay(300);
  Serial.printf("\n=== RADIO EQUIPO %d %s===\n", EQUIPO, ES_BASE ? "(BASE) " : "");
  pinMode(LED_PIN, OUTPUT);
  pinMode(BOTON_SOS, INPUT_PULLUP);
  siguienteId = esp_random() % 50000 + 1;         // al azar: evita repetidos tras reiniciar

  // Memoria para fotos y audios (se borra al encender)
  if (!LittleFS.begin(true)) Serial.println("!! Error de memoria LittleFS");
  File dir = LittleFS.open("/m");
  if (dir) {
    std::vector<String> borrar;
    for (File f = dir.openNextFile(); f; f = dir.openNextFile()) borrar.push_back(String("/m/") + f.name());
    for (auto& r : borrar) LittleFS.remove(r);
  } else LittleFS.mkdir("/m");

  loraIniciar();

  // WiFi: punto de acceso para el celular (+ conexión a casa si es BASE)
  WiFi.mode(WIFI_AP_STA);
  uint8_t canal = CANAL_WIFI;
#if ES_BASE
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Conectando al WiFi de casa");
  for (int i = 0; i < 60 && WiFi.status() != WL_CONNECTED; i++) { delay(250); Serial.print("."); }
  Serial.println();
  if (WiFi.status() == WL_CONNECTED) {
    canal = WiFi.channel();
    Serial.printf("WiFi OK. >>> CANAL %d: ponlo en CANAL_WIFI de la otra placa\n", canal);
  } else Serial.println("Sin WiFi de casa: la radio funciona igual, sin la web");
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMqtt);
  mqtt.setBufferSize(512);
#endif
  String nombre = "Radio-" + String(EQUIPO);
  WiFi.softAP(nombre.c_str(), CLAVE_AP, canal);
  esp_wifi_set_ps(WIFI_PS_NONE);
  Serial.printf("WiFi para el celular: %s / %s -> abre http://192.168.4.1\n", nombre.c_str(), CLAVE_AP);

  // ESP-NOW
  colaEsp = xQueueCreate(24, sizeof(RxEsp));
  if (esp_now_init() == ESP_OK) {
    esp_now_register_recv_cb(onEspNow);
```

```
    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, BCAST, 6);
    peer.channel = 0;
    peer.ifidx = WIFI_IF_AP;
    peer.encrypt = false;
    esp_now_add_peer(&peer);
  } else Serial.println("!! Error iniciando ESP-NOW");

  iniciarWeb();
  Serial.println("Listo.");
}
```

13) loop()

Se repite miles de veces por segundo:

- Atiende la página web y el DNS.
- Procesa todo lo que llegó por ESP-NOW (mensajes o trozos de archivo).
- Atiende el LoRa, los reintentos y las transferencias.
- Envía latidos: cada 2 s por ESP-NOW y cada ~30 s por LoRa. El equipo 2 espera 1.5 s más que el 1, para que no transmitan siempre al mismo tiempo.
- Vigila el botón BOOT (2 s = SOS) y hace parpadear el LED si hay una alarma.
- Lo que escribas en el Monitor Serie también se envía como mensaje: sirve para probar sin celular.

```
// ---------------------------------------------------------------------------
// 13) LOOP
// ---------------------------------------------------------------------------
void loop() {
  dns.processNextRequest();
  server.handleClient();

  // Lo que llegó por ESP-NOW
  RxEsp r;
  while (xQueueReceive(colaEsp, &r, 0) == pdTRUE) {
    bool antes = cerca();
    ultimoEspNow = millis();
    espRssi = r.rssi;
    if (!antes) version++;
    char c = r.d[0];
    if (c == 'f' || c == 'd' || c == 'a') procesarArchivo(r.d, r.len, r.rssi);
    else procesarTramaTexto(String((const char*)r.d, r.len), 'E', r.rssi);
  }

  loraLoop();
  revisarPendientes();
  txLoop();
  rxLoop();

  // Latidos: "estoy aquí"
  static uint32_t latidoEsp = 0, latidoLora = 0;
  if (millis() - latidoEsp > 2000) { latidoEsp = millis(); espEnviarTexto("P," + String(EQUIPO)); }
  if (millis() - latidoLora > 30000 + EQUIPO * 1500) { latidoLora = millis(); loraEnviar("P," + String(EQUIP
O)); }

  // ¿Se perdió el enlace cercano? avisar a la web
  static bool cercaAntes = false;
  if (cerca() != cercaAntes) { cercaAntes = cerca(); version++; }

  // Botón BOOT: mantener 2 s = SOS
  static uint32_t pulsado = 0;
  if (digitalRead(BOTON_SOS) == LOW) {
    if (!pulsado) pulsado = millis();
    else if (pulsado != 1 && millis() - pulsado > 2000) {
      enviarMensaje(SOS, "SOS: botón de emergencia");
```

```
      pulsado = 1;                                  // no repetir hasta soltar
    }
  } else pulsado = 0;

  // LED: parpadeo rápido si hay un SOS sin atender
  if (sosDe && millis() < sosHasta) digitalWrite(LED_PIN, (millis() / 120) % 2);
  else if (sosDe) { sosDe = 0; digitalWrite(LED_PIN, LOW); }

  // Mensajes escritos en el Monitor Serie
  static String linea;
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') { if (linea.length()) enviarMensaje(TEXTO, linea); linea = ""; }
    else linea += c;
  }

#if ES_BASE
  mqttLoop();
#endif
}
```

10 Código de la página del chat

Archivo `pagina.h`, **completo**. Es HTML, CSS y JavaScript guardados dentro del ESP32.

Inicio del archivo

La página completa se guarda como una sola cadena de texto en la memoria de programa (`PROGMEM`). `R"PAGINA( ... )PAGINA"` permite escribir HTML con comillas y saltos de línea sin escaparlos. La etiqueta `viewport` hace que se vea bien en el celular.

```
// Página del chat que el ESP32 sirve al celular (http://192.168.4.1)
// Se guarda en la memoria de programa (PROGMEM) para no gastar RAM.
#pragma once
const char PAGINA[] PROGMEM = R"PAGINA(
<!doctype html><html lang="es"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1">
<title>Radio</title>
```

Estilos (CSS)

Tema oscuro tipo app de mensajería: la página ocupa toda la pantalla, la lista de mensajes se desplaza y la barra de escribir queda fija abajo. Tus mensajes van a la derecha en azul; los SOS en rojo con un borde.

```
<style>
:root{--bg:#0f1720;--card:#17212c;--line:#253241;--tx:#e8eef4;--mu:#8a9bab;--yo:#1e4a6e;--ok:#3fd17a;--bad:#ff5d5d;--lo:#ffb23b}
*{box-sizing:border-box}html,body{height:100%}
body{margin:0;background:var(--bg);color:var(--tx);font:15px/1.4 system-ui,-apple-system,Roboto,sans-serif;display:flex;flex-direction:column}
header{padding:10px 12px;border-bottom:1px solid var(--line);display:flex;flex-direction:column;gap:8px}
.top{display:flex;align-items:center;justify-content:space-between}
h1{font-size:17px;margin:0}
.chips{display:flex;gap:6px;flex-wrap:wrap}
.chip{font-size:12px;padding:4px 9px;border-radius:99px;background:var(--line);color:var(--mu)}
.chip.on{background:rgba(63,209,122,.15);color:var(--ok)}
.chip.lo{background:rgba(255,178,59,.15);color:var(--lo)}
#sosBtn{background:var(--bad);color:#fff;border:0;border-radius:10px;padding:7px 12px;font-weight:700}
#alerta{display:none;background:var(--bad);color:#fff;padding:12px;font-weight:700;justify-content:space-between;align-items:center;animation:p 1s infinite}
#alerta button{background:#fff;color:var(--bad);border:0;border-radius:8px;padding:6px 12px;font-weight:700}
@keyframes p{50%{opacity:.75}}
#lista{flex:1;overflow-y:auto;padding:12px;display:flex;flex-direction:column;gap:8px}
.m{max-width:82%;padding:8px 11px;border-radius:14px;background:var(--card);word-wrap:break-word}
.m.yo{align-self:flex-end;background:var(--yo)}
.m.sos{background:#5c1a1a;border:1px solid var(--bad)}
.m small{display:block;font-size:11px;color:var(--mu);margin-top:3px}
.m img{max-width:100%;border-radius:8px;display:block}
.m audio{width:230px;max-width:100%}
.vacio{color:var(--mu);text-align:center;margin:auto;font-size:14px}
form{display:flex;gap:6px;padding:8px;border-top:1px solid var(--line)}
input[type=text]{flex:1;min-width:0;background:var(--card);border:1px solid var(--line);color:var(--tx);border-radius:12px;padding:11px;font:inherit}
.b{background:var(--card);border:1px solid var(--line);color:var(--tx);border-radius:12px;padding:0 12px;font-size:18px}
.b:disabled{opacity:.35}
#env{background:#3ba7ff;color:#04121f;border:0;font-weight:700;font-size:15px}
#aviso{display:none;position:fixed;left:12px;right:12px;bottom:70px;background:#000c;padding:10px 12px;border-radius:10px;font-size:14px}
</style></head><body>
```

Estructura (HTML)

Encabezado con el número de equipo, el botón SOS y tres “chips” de estado (cerca, LoRa y web). Debajo: la franja de alarma, la lista de mensajes, la barra con 📷, 🎤, el texto y Enviar. Los dos `input type=file` ocultos abren la cámara y la grabadora del celular.

```
<header>
  <div class="top"><h1 id="tit">Radio</h1><button id="sosBtn">SOS</button></div>
  <div class="chips"><span id="cCerca" class="chip">⚡ Cerca: —</span><span id="cLora" class="chip">📡 LoRa:
—</span><span id="cWeb" class="chip" style="display:none">🌐 Web</span></div>
</header>
<div id="alerta"><span id="alertaTxt">🚨 SOS</span><button id="visto">Visto</button></div>
<div id="lista"><div class="vacio" id="vacio">Aún no hay mensajes.<br>Escribe abajo para empezar.</div></div
>
<div id="aviso"></div>
<form id="f">
  <button type="button" class="b" id="bFoto" title="Foto">📷</button>
  <button type="button" class="b" id="bAudio" title="Nota de voz">🎤</button>
  <input type="text" id="txt" maxlength="160" placeholder="Mensaje…" autocomplete="off">
  <button class="b" id="env">Enviar</button>
</form>
<input type="file" id="iFoto" accept="image/*" capture="environment" hidden>
<input type="file" id="iAudio" accept="audio/*" capture hidden>
<script>
```

Utilidades

`$()` busca un elemento por su id. `aviso()` muestra un mensaje flotante. `hora()` anota la hora en que la página vio cada mensaje por primera vez (el ESP32 no tiene reloj).

```
// --- utilidades ---
const $ = id => document.getElementById(id);
let estado = null, vistos = {}, nodos = {};

function aviso(t, ms = 3500) { const a = $("aviso"); a.textContent = t; a.style.display = "block"; clearTime
out(a._t); a._t = setTimeout(() => a.style.display = "none", ms); }
function nombre(n) { return n == 0 ? "Web" : "Equipo " + n; }
function hora(k) { if (!vistos[k]) vistos[k] = new Date(); return vistos[k].toLocaleTimeString("es-PA", { ho
ur: "2-digit", minute: "2-digit" }); }
```

Cómo se dibuja cada mensaje

`pie()` arma la línea pequeña: quién, hora, vía (⚡ cerca / 📡 LoRa) y estado (⏳, ✓ entregado, ✕). `crear()` genera la burbuja: texto, foto (`img`) o nota de voz (`audio` con controles). Usa `textContent` para que un mensaje no pueda inyectar código en la página.

```
// --- cómo se dibuja cada mensaje ---
function pie(m) {
  const via = m.via == "E" ? "⚡ cerca" : m.via == "L" ? "📡 LoRa" : "";
  let est = "";
  if (m.mio) est = m.e == 0 ? (m.t >= 2 ? "⏳ " + m.p + "%" : "⏳ enviando") : m.e == 1 ? "✓ entregado" : "✕
no entregado";
  else est = m.rssi ? m.rssi + " dBm" : "";
  return [m.mio ? "Tú" : nombre(m.de), hora(m.mio + "-" + m.de + "-" + m.id), via, est].filter(Boolean).join
(" · ");
}

function crear(m) {
  const d = document.createElement("div");
  d.className = "m" + (m.mio ? " yo" : "") + (m.t == 1 ? " sos" : "");
  if (m.t == 3) { const i = document.createElement("img"); i.src = m.x; i.onload = () => { const l = $("list
a"); l.scrollTop = l.scrollHeight; }; d.append(i); }
  else if (m.t == 2) { const a = document.createElement("audio"); a.controls = true; a.preload = "none"; a.s
```

```
rc = m.x; d.append(a); }
  else { const s = document.createElement("div"); s.textContent = (m.t == 1 ? "🚨 " : "") + m.x; d.append
(s); }
  const p = document.createElement("small"); d.append(p); d._pie = p;
  return d;
}
```

Dibujar el estado completo

`pintar()` actualiza los chips, activa o desactiva 📷 y 🎤 (solo funcionan cerca), muestra la alarma SOS y hace vibrar el celular. Cada mensaje se dibuja **una sola vez** y luego solo se actualiza su pie; así un audio que estás escuchando no se interrumpe.

```
// --- dibujar el estado completo ---
function pintar(j) {
  $("tit").textContent = "Radio · Equipo " + j.yo;
  const c = $("cCerca"); c.textContent = "⚡ Cerca: " + (j.cerca.on ? j.cerca.rssi + " dBm" : "fuera de alcan
ce"); c.className = "chip" + (j.cerca.on ? " on" : "");
  const l = $("cLora");
  l.textContent = "📡 LoRa: " + (!j.lora.ok ? "sin módulo" : j.lora.on ? j.lora.rssi + " dBm · SNR " + j.lor
a.snr : "esperando…");
  l.className = "chip" + (j.lora.on ? " lo" : "");
  if ("web" in j) { const w = $("cWeb"); w.style.display = ""; w.className = "chip" + (j.web ? " on" : "");
w.textContent = "🌐 Web: " + (j.web ? "conectada" : "sin conexión"); }
  const medios = j.cerca.on && !j.ocupado;
  $("bFoto").disabled = $("bAudio").disabled = !medios;

  const al = $("alerta");
  if (j.sos) { $("alertaTxt").textContent = "🚨 SOS de " + nombre(j.sos); al.style.display = "flex"; if (nav
igator.vibrate) navigator.vibrate([300, 150, 300]); }
  else al.style.display = "none";

  const lista = $("lista"), vivos = {};
  let abajo = lista.scrollHeight - lista.scrollTop - lista.clientHeight < 60;
  for (const m of j.msgs) {
    const k = m.mio + "-" + m.de + "-" + m.id; vivos[k] = 1;
    if (!nodos[k]) { nodos[k] = crear(m); lista.append(nodos[k]); abajo = true; }
    nodos[k]._pie.textContent = pie(m);
  }
  for (const k in nodos) if (!vivos[k]) { nodos[k].remove(); delete nodos[k]; }
  $("vacio").style.display = j.msgs.length ? "none" : "";
  if (abajo) lista.scrollTop = lista.scrollHeight;
}
```

Preguntar a la placa cada segundo

`actualizar()` pide `/api`, dibuja y se vuelve a programar en 1 s. Si la placa no responde, avisa “Sin conexión con la placa”.

```
// --- preguntar a la placa cada segundo ---
async function actualizar() {
  try { const r = await fetch("/api"); estado = await r.json(); pintar(estado); }
  catch (e) { $("cCerca").textContent = "⚠ Sin conexión con la placa"; }
  setTimeout(actualizar, 1000);
}
```

Enviar texto y SOS

Envía el formulario a `/enviar`. El botón SOS pide confirmación antes de mandar la alarma.

```
// --- enviar texto y SOS ---
function post(url, datos) { return fetch(url, { method: "POST", body: new URLSearchParams(datos) }); }

$("f").onsubmit = e => {
```

```
  e.preventDefault();
  const t = $("txt").value.trim(); if (!t) return;
  post("/enviar", { t }); $("txt").value = "";
};
$("sosBtn").onclick = () => { if (confirm("¿Enviar SOS a la otra estación?")) post("/enviar", { sos: 1, t:
"" }); };
$("visto").onclick = () => post("/sosvisto", {});
```

Fotos y notas de voz

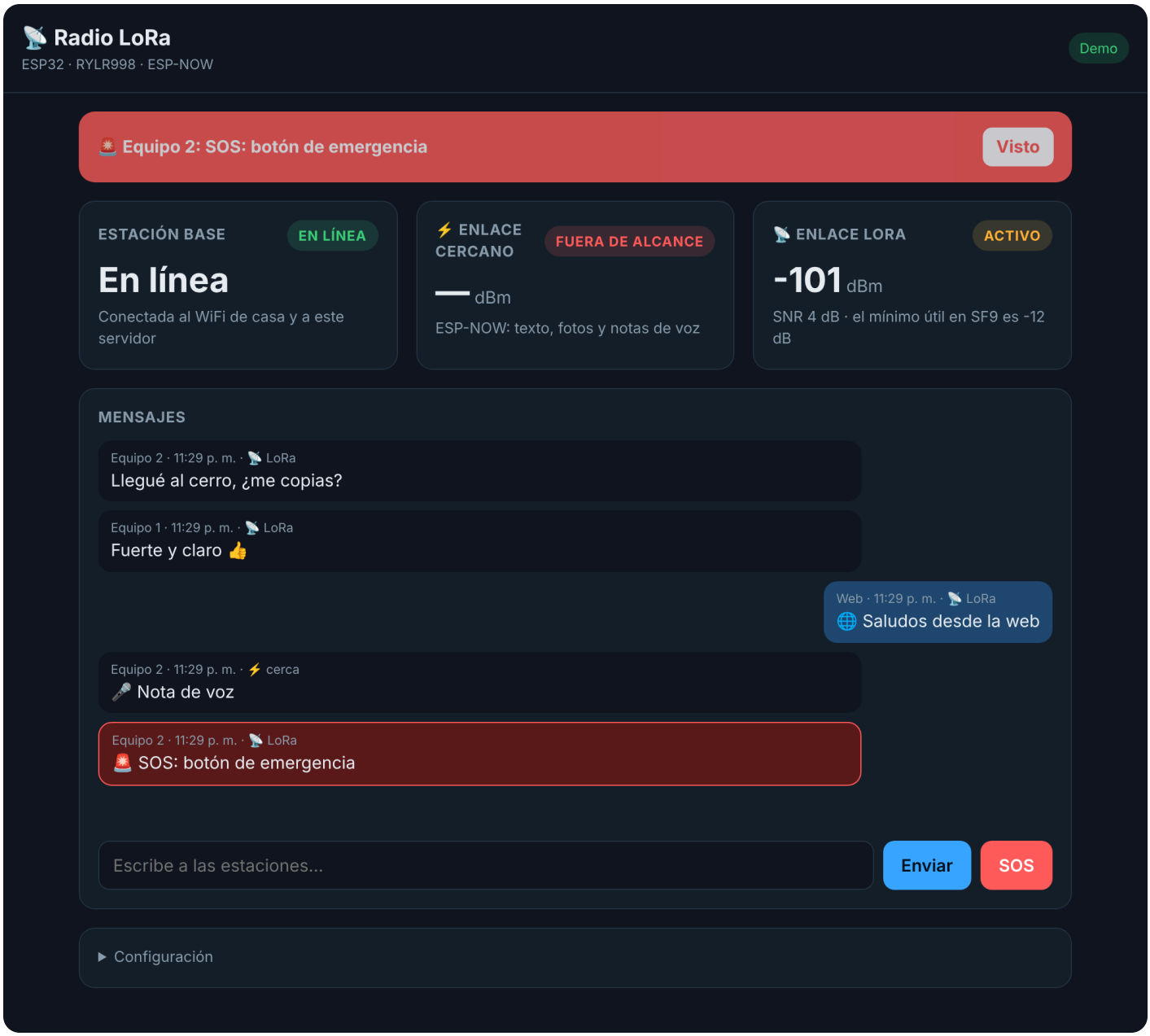
`subir()` manda el archivo con `XMLHttpRequest` para mostrar el porcentaje de carga. La foto se **reduce a 800 px y se comprime** en el navegador con un `canvas` (queda en 40–80 KB). La nota de voz se graba con la app del celular y se envía tal cual; por eso conviene que sea corta (máx. 400 KB).

```
// --- fotos y notas de voz ---
function subir(blob, tipo, ext) {
  if (blob.size > 400000) { aviso("Muy grande (" + Math.round(blob.size / 1024) + " KB). Máximo 400 KB: grab
a menos tiempo."); return; }
  const fd = new FormData(); fd.append("archivo", blob, "a." + ext);
  const x = new XMLHttpRequest();
  x.open("POST", "/subir?tipo=" + tipo + "&ext=" + ext);
  x.upload.onprogress = e => aviso("Cargando… " + Math.round(e.loaded * 100 / e.total) + "%", 2000);
  x.onload = () => aviso(x.status == 200 ? "Enviando por radio…" : x.responseText);
  x.onerror = () => aviso("Error al cargar el archivo");
  x.send(fd);
}

$("bFoto").onclick = () => $("iFoto").click();
$("bAudio").onclick = () => $("iAudio").click();
$("iFoto").onchange = e => {
  const f = e.target.files[0]; e.target.value = ""; if (!f) return;
  const img = new Image();
  img.onload = () => {   // reducir la foto a 800 px y comprimir en JPEG
    const k = Math.min(1, 800 / Math.max(img.width, img.height));
    const c = document.createElement("canvas"); c.width = img.width * k; c.height = img.height * k;
    c.getContext("2d").drawImage(img, 0, 0, c.width, c.height);
    c.toBlob(b => subir(b, "foto", "jpg"), "image/jpeg", 0.6);
    URL.revokeObjectURL(img.src);
  };
  img.src = URL.createObjectURL(f);
};
$("iAudio").onchange = e => {
  const f = e.target.files[0]; e.target.value = ""; if (!f) return;
  let ext = (f.name.split(".").pop() || "m4a").toLowerCase().slice(0, 5);
  subir(f, "audio", ext);
};
actualizar();
</script></body></html>
)PAGINA";
```

11 Página web en GitHub

Solo funciona si una placa está configurada como **base** (`ES_BASE 1`) y conectada a tu WiFi.



Archivo `web/index.html` abierto con `?demo` .

Publicarla

1. En github.com crea un repositorio público, por ejemplo `radio-lora` .
2. **Add file → Upload files** y sube `index.html` . **Commit changes**.
3. **Settings → Pages** → Branch **main**, carpeta **/ (root)** → **Save**.
4. En 1–2 minutos estará en `https://TU-USUARIO.github.io/radio-lora/` . En **Configuración** pon el mismo `TOPIC_PREFIX` del ESP32.

Privacidad: el servidor MQTT público no tiene contraseña. Lo que viaja por internet (textos y estado) puede verlo quien adivine tu prefijo. Lo que viaja por radio entre estaciones sí va cifrado con `LORA_CLAVE` en LoRa (ESP-NOW va sin cifrar).

Código de la página, explicado

Encabezado

Página para GitHub Pages. Carga **MQTT.js**, la librería que conecta el navegador con el servidor MQTT.

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="robots" content="noindex, nofollow">
<title>Radio LoRa</title>
<script src="https://unpkg.com/mqtt@5/dist/mqtt.min.js"></script>
```

Estilos

Mismos colores que el chat del celular. Tres tarjetas de estado que se acomodan solas en PC o celular.

```
<style>
  :root{--bg:#0f1720;--card:#17212c;--line:#253241;--tx:#e8eef4;--mu:#8a9bab;--yo:#1e4a6e;--ok:#3fd17a;--bad:#ff5d5d;--lo:#ffb23b;--az:#3ba7ff}
  *{box-sizing:border-box}
  body{margin:0;background:var(--bg);color:var(--tx);font:15px/1.45 system-ui,-apple-system,"Segoe UI",Roboto,sans-serif}
  header{display:flex;align-items:center;justify-content:space-between;gap:12px;padding:16px;border-bottom:1px solid var(--line)}
  h1{font-size:19px;margin:0}
  h1 small{display:block;font-size:12px;color:var(--mu);font-weight:400}
  .pill{font-size:12px;padding:5px 10px;border-radius:99px;background:var(--line);color:var(--mu);white-space:nowrap}
  .pill.on{background:rgba(63,209,122,.15);color:var(--ok)}
  .pill.lo{background:rgba(255,178,59,.15);color:var(--lo)}
  .pill.off{background:rgba(255,93,93,.15);color:var(--bad)}
  main{max-width:900px;margin:0 auto;padding:16px;display:grid;grid-template-columns:minmax(0,1fr);gap:16px}
  .grid{min-width:0}
  .grid{display:grid;gap:16px;grid-template-columns:repeat(auto-fit,minmax(240px,1fr))}
  .card{background:var(--card);border:1px solid var(--line);border-radius:14px;padding:16px}
  .card h2{margin:0 0 10px;font-size:13px;color:var(--mu);text-transform:uppercase;letter-spacing:.6px;display:flex;justify-content:space-between;align-items:center}
  .big{font-size:30px;font-weight:700;line-height:1.1}
  .big span{font-size:15px;color:var(--mu);font-weight:400}
  .sub{font-size:13px;color:var(--mu);margin-top:6px}
  #alerta{display:none;background:var(--bad);color:#fff;border-radius:14px;padding:14px 16px;font-weight:700;align-items:center;justify-content:space-between;animation:p 1s infinite}
  #alerta button{background:#fff;color:var(--bad);border:0;border-radius:8px;padding:6px 12px;font-weight:700}
  @keyframes p{50%{opacity:.75}}
  #log{height:340px;overflow-y:auto;display:flex;flex-direction:column;gap:8px;margin-bottom:10px}
  .m{max-width:80%;padding:8px 11px;border-radius:12px;background:var(--bg)}
  .m small{display:block;font-size:11px;color:var(--mu)}
  .m.web{align-self:flex-end;background:var(--yo)}
  .m.sos{background:#5c1a1a;border:1px solid var(--bad)}
  .vacio{color:var(--mu);margin:auto;font-size:14px}
  form{display:flex;gap:8px}
  input{flex:1;min-width:0;width:0;background:var(--bg);border:1px solid var(--line);color:var(--tx);border-radius:10px;padding:10px 12px;font:inherit}
  button{border:0;border-radius:10px;padding:10px 16px;font:inherit;font-weight:600;cursor:pointer}
  #env{background:var(--az);color:#04121f}
```

```
  #sos{background:var(--bad);color:#fff}
  details{font-size:13px;color:var(--mu)}
  details input{margin-top:6px;width:100%}
</style>
```

Estructura

Franja de alarma SOS, tarjetas (base, enlace cercano, enlace LoRa), chat con botones Enviar y SOS, y el panel de configuración.

```
</head>
<body>
<header>
  <h1>📡 Radio LoRa<small>ESP32 · RYLR998 · ESP-NOW</small></h1>
  <span id="conn" class="pill">Conectando…</span>
</header>

<main>
  <div id="alerta"><span id="alertaTxt">🚨 SOS</span><button id="visto">Visto</button></div>

  <div class="grid">
    <section class="card">
      <h2>Estación base <span id="stBase" class="pill">—</span></h2>
      <div class="big" id="baseTxt">—</div>
      <div class="sub" id="baseSub">Esperando datos de la base…</div>
    </section>
    <section class="card">
      <h2>⚡ Enlace cercano <span id="stCerca" class="pill">—</span></h2>
      <div class="big" id="cercaRssi">—<span> dBm</span></div>
      <div class="sub">ESP-NOW: texto, fotos y notas de voz</div>
    </section>
    <section class="card">
      <h2>📡 Enlace LoRa <span id="stLora" class="pill">—</span></h2>
      <div class="big" id="loraRssi">—<span> dBm</span></div>
      <div class="sub" id="loraSnr">SNR —</div>
    </section>
  </div>

  <section class="card">
    <h2>Mensajes</h2>
    <div id="log"><div class="vacio" id="vacio">Sin mensajes todavía</div></div>
    <form id="f">
      <input id="txt" maxlength="150" placeholder="Escribe a las estaciones…" autocomplete="off">
      <button id="env">Enviar</button>
      <button type="button" id="sos">SOS</button>
    </form>
  </section>

  <details class="card">
    <summary>Configuración</summary>
    <label>Prefijo (igual al TOPIC_PREFIX del ESP32)<input id="prefix"></label>
  </details>
</main>

<script>
```

Configuración

Dirección del servidor MQTT por WebSocket seguro (necesario porque GitHub Pages usa HTTPS) y el prefijo por defecto. Abriendo la página con `?demo` se ve con datos de ejemplo.

```
const BROKER = "wss://broker.hivemq.com:8884/mqtt";
const DEFAULT_PREFIX = "panarepara-radio-acbdcce2";
const DEMO = new URLSearchParams(location.search).has("demo");
const $ = id => document.getElementById(id);
```

Prefijo guardado

El prefijo se puede cambiar en Configuración y queda guardado en ese navegador.

```
// --- prefijo guardado en el navegador ---
let prefix = DEFAULT_PREFIX;
try { prefix = localStorage.getItem("prefix") || DEFAULT_PREFIX; } catch (e) {}
$("prefix").value = prefix;
$("prefix").addEventListener("change", () => {
  try { localStorage.setItem("prefix", $("prefix").value.trim()); } catch (e) {}
  location.reload();
});
```

Estado de los enlaces

`mostrarEstado()` pinta las tarjetas con lo que la base publica cada 10 s. Si pasan 30 s sin noticias, la base se marca “Sin datos”.

```
// --- estado de los enlaces ---
let ultimoEstado = 0;
function pill(el, on, txtOn, txtOff, cls = "on") { el.textContent = on ? txtOn : txtOff; el.className = "pil
l " + (on ? cls : "off"); }
function mostrarEstado(e, vivo) {
  if (vivo) ultimoEstado = Date.now();
  if (!e.base) { pill($("stBase"), false, "", "Apagada"); $("baseTxt").textContent = "Apagada"; return; }
  $("baseTxt").textContent = "En línea";
  $("baseSub").textContent = "Conectada al WiFi de casa y a este servidor";
  pill($("stCerca"), e.cerca.on, "Cerca", "Fuera de alcance");
  $("cercaRssi").innerHTML = (e.cerca.on ? e.cerca.rssi : "—") + "<span> dBm</span>";
  pill($("stLora"), e.lora.on, "Activo", "Sin señal", "lo");
  $("loraRssi").innerHTML = (e.lora.on ? e.lora.rssi : "—") + "<span> dBm</span>";
  $("loraSnr").textContent = "SNR " + (e.lora.on ? e.lora.snr + " dB" : "—") + " · el mínimo útil en SF9 es
-12 dB";
}
setInterval(() => {   // la base publica su estado cada 10 s
  const on = Date.now() - ultimoEstado < 30000;
  pill($("stBase"), on, "En línea", "Sin datos");
}, 1000);
```

Mensajes

`addMsg()` dibuja cada mensaje y, si es un SOS de una estación, muestra la franja roja y hace vibrar el celular.

```
// --- mensajes ---
function nombre(n) { return n == 0 ? "Web" : "Equipo " + n; }
function addMsg(m) {
  $("vacio").style.display = "none";
  const d = document.createElement("div");
  d.className = "m" + (m.de == 0 ? " web" : "") + (m.t == 1 ? " sos" : "");
  const s = document.createElement("small");
  const via = m.via == "E" ? " · ⚡ cerca" : m.via == "L" ? " · 📡 LoRa" : "";
  s.textContent = nombre(m.de) + " · " + new Date().toLocaleTimeString("es-PA", { hour: "2-digit", minute:
"2-digit" }) + via;
  d.append(s, document.createTextNode((m.t == 1 ? "🚨 " : "") + m.x));
  $("log").append(d); $("log").scrollTop = $("log").scrollHeight;
  if (m.t == 1 && m.de != 0) {
    $("alertaTxt").textContent = "🚨 " + nombre(m.de) + ": " + m.x;
    $("alerta").style.display = "flex";
    if (navigator.vibrate) navigator.vibrate([300, 150, 300]);
  }
}
$("visto").onclick = () => $("alerta").style.display = "none";
```

Conexión MQTT

Se suscribe a todos los temas del prefijo. Enviar publica en `web/msg` y SOS en `web/sos`; la base los transmite por radio y los vuelve a publicar, por eso aparecen en el chat.

```
// --- conexión MQTT ---
let client = null;
if (!DEMO) {
  client = mqtt.connect(BROKER, { clientId: "web-" + Math.random().toString(16).slice(2, 10) });
  client.on("connect", () => { pill($("conn"), true, "Conectado", ""); client.subscribe(prefix + "/#"); });
  client.on("close", () => pill($("conn"), false, "", "Desconectado"));
  client.on("message", (topic, payload, packet) => {
    const sub = topic.slice(prefix.length + 1);
    try {
      if (sub === "estado") mostrarEstado(JSON.parse(payload), !packet.retain);
      else if (sub === "radio/msg") addMsg(JSON.parse(payload));
    } catch (e) { console.warn(e); }
  });
}
// La propia base re-publica lo que enviamos (como "Web"), así que no lo dibujamos aquí
$("f").onsubmit = e => {
  e.preventDefault();
  const t = $("txt").value.trim(); if (!t) return;
  if (client) client.publish(prefix + "/web/msg", t);
  $("txt").value = "";
};
$("sos").onclick = () => {
  if (!confirm("¿Enviar SOS a las estaciones?")) return;
  if (client) client.publish(prefix + "/web/sos", "SOS desde la web");
};
```

Modo demostración

Datos de ejemplo para ver la página sin hardware.

```
// --- modo demostración (?demo) ---
if (DEMO) {
  pill($("conn"), true, "Demo", "");
  mostrarEstado({ base: true, cerca: { on: false, rssi: 0 }, lora: { on: true, rssi: -101, snr: 4 } }, tru
e);
  addMsg({ de: 2, t: 0, via: "L", x: "Llegué al cerro, ¿me copias?" });
  addMsg({ de: 1, t: 0, via: "L", x: "Fuerte y claro 👍" });
  addMsg({ de: 0, t: 0, via: "L", x: "🌐 Saludos desde la web" });
  addMsg({ de: 2, t: 2, via: "E", x: "🎤 Nota de voz" });
  addMsg({ de: 2, t: 1, via: "L", x: "SOS: botón de emergencia" });
}
</script>
</body>
</html>
```

12 Pruebas de alcance

1. **En la mesa** (las dos placas juntas): el chip ⚡ debe marcar −30 a −50 dBm y los mensajes llegar al instante.
2. **Salir de ESP-NOW**: aléjate hasta que ⚡ diga “fuera de alcance”. Los mensajes seguirán llegando, ahora con 📡 LoRa (tardan 1–3 s).
3. **Prueba LoRa**: sigue alejándote y anota dónde está, RSSI y SNR. Mientras el SNR sea mayor que −10 dB el enlace es seguro; entre −10 y −15 dB empezarán los fallos.
4. **Prueba SOS**: mantén BOOT 2 s en una placa; la otra debe parpadear y mostrar la alarma.

Señal LoRa (RSSI)	Significado
−40 a −90 dBm	Excelente
−90 a −110 dBm	Buena
−110 a −125 dBm	Al límite: mira el SNR

13 Solución de problemas

Síntoma	Solución
“El RYLR998 no responde”	Revisa TXD→16 y RXD→17 (cruzados), 3V3 y GND. Confirma que el módulo sea un RYLR998.
LoRa “esperando…” siempre	Los dos deben tener la misma `LORA_RED`, `LORA_BANDA`, `LORA_PARAM` y `LORA_CLAVE`, y `EQUIPO` distinto. Espera 1 minuto (el latido LoRa es cada ~30 s).
⚡ nunca dice “cerca”	Canales distintos: pon en `CANAL_WIFI` el canal que imprime la base.
El chat no abre	Escribe `http://192.168.4.1` (con http), desactiva los datos móviles y acepta “mantener conexión”.
📷 / 🎤 deshabilitados	Solo funcionan cerca, y uno a la vez.
“Archivo muy grande”	Graba una nota de voz más corta (máx. 400 KB).
El audio no suena en el otro celular	Algunos formatos (por ejemplo .amr) no los reproduce todo navegador. Prueba otra app de grabación que guarde .m4a.
La web dice “Sin datos”	La base debe tener `ES_BASE 1`, tu WiFi correcto y el mismo prefijo que la página.

14 Límites y próximos pasos

- **La voz en tiempo real no es posible por LoRa**: el RYLR998 transmite unos pocos cientos de bytes por segundo, y la voz necesita miles. Las notas de voz y fotos solo viajan por ESP-NOW, de cerca.
- **Walkie-talkie real de cerca**: agregando a cada placa un micrófono INMP441, un amplificador MAX98357A y un parlante (unos $10 por estación) se puede sumar la voz en tiempo real por ESP-NOW de la primera guía.
- **Más alcance**: subir las antenas (techo, mástil) o usar antenas de 915 MHz con más ganancia.
- **Más sensores**: un BME280 o un GPS (NEO-6M) en la estación de campo para enviar clima o ubicación por LoRa cada pocos minutos.
- **Legalidad**: antes de usarlo, confirma con la ASEP (o la autoridad de tu país) que la frecuencia de 915 MHz y la potencia de 22 dBm están permitidas para uso libre.

15 Proceso del sistema

Registro de cómo se desarrolló el sistema, desde las piezas iniciales hasta la página publicada en el sitio de PanaRepara. Fecha: 28 de octubre de 2026.

1. Punto de partida

> **18:45 · Identificación del hardware.** A partir de la foto inicial se identificaron 2 ESP32 DevKit y dos módulos de radio. Se aclaró que un HC-12 y un RYLR998 no se comunican entre sí; se confirmó que el equipo dispone de **2 × RYLR998**.
>
> **19:00 · Primer diseño.** Walkie-talkie con estación meteorológica (guía anterior). Se dejó para más adelante porque requiere micrófono, amplificador, parlante y sensor que aún no se tienen.
>
> **19:24 · Diseño final.** Radio híbrida que usa solo lo disponible: ESP-NOW para cerca (texto, fotos, voz grabada), LoRa para lejos (texto y SOS), chat desde el celular y panel web.

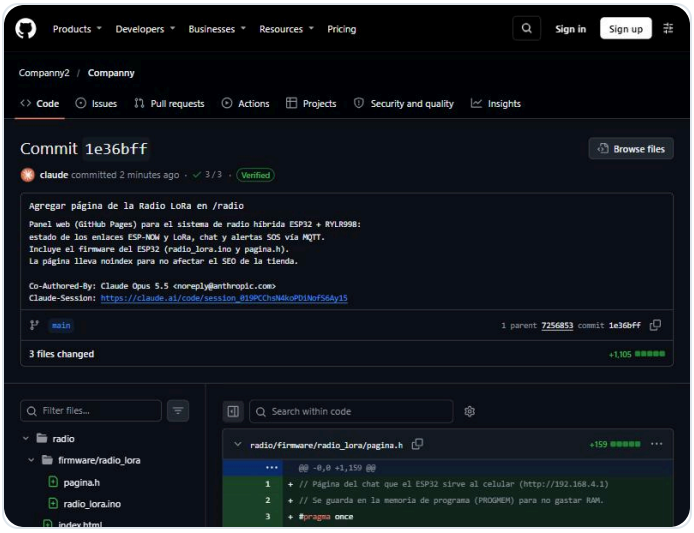
2. Desarrollo y verificación

> **Firmware.** `radio_lora.ino` + `pagina.h`. Se compiló con el núcleo ESP32 3.3.2 en sus dos variantes (base y remota): sin errores ni advertencias, 77 % de la memoria de programa y 13 % de la RAM.
>
> **Revisión del protocolo LoRa.** Se contrastó con la guía oficial de comandos AT de REYAX (rango de SF, NETWORKID 18, preámbulo 12, clave CPIN de 8 dígitos, máximo 240 bytes). Se reforzó la lectura de `+RCV` para tolerar espacios entre campos.
>
> **Interfaces.** El chat del celular y el panel web se probaron en un navegador con datos simulados, sin errores de JavaScript (capturas en las secciones 8 y 11).

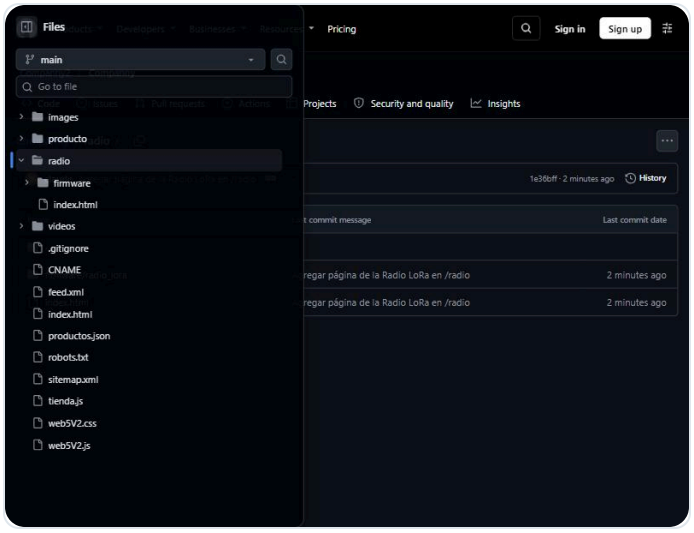
3. Publicación en GitHub

> **Repositorio.** Se usó el repositorio existente `Companny2/Companny`, que publica el sitio **panarepara.com** con GitHub Pages. Para no tocar la tienda, todo se agregó en una carpeta nueva `/radio`.
>
> **Seguridad.** Se generó un prefijo MQTT aleatorio (`panarepara-radio-acbdcce2`), igual en el firmware y en la web, y la página lleva `noindex` para que Google no la mezcle con los productos de la tienda.
>
> **23:29 · Commit `1e36bff`.** "Agregar página de la Radio LoRa en /radio": 3 archivos, 1 105 líneas.

Archivo en el repositorio	Qué es
`radio/index.html`	Panel web → panarepara.com/radio/
`radio/firmware/radio_lora/radio_lora.ino`	Programa del ESP32
`radio/firmware/radio_lora/pagina.h`	Chat que el ESP32 sirve al celular
`radio/Guia_Radio_LoRa_ESP32.pdf`	Esta guía



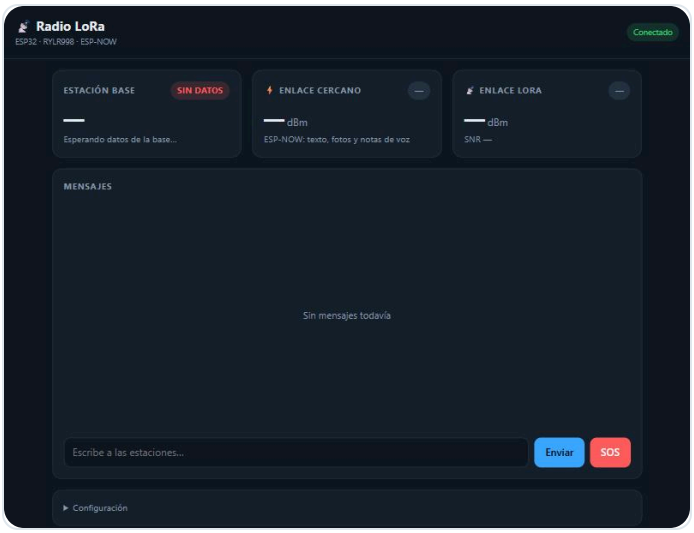
El commit en GitHub con los archivos agregados.



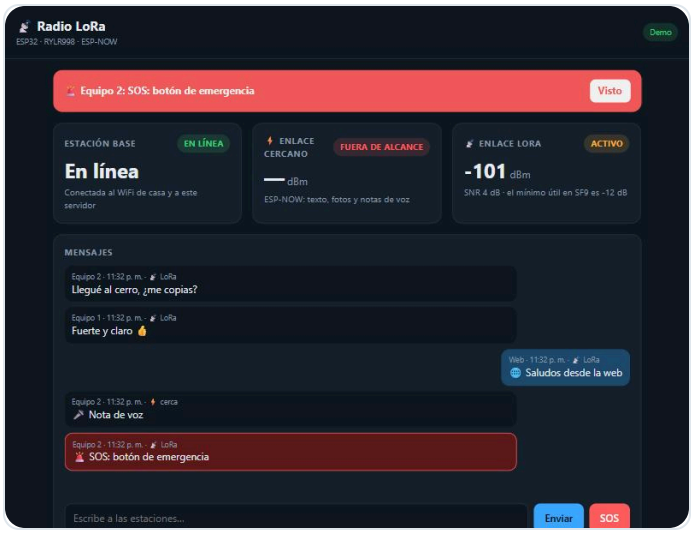
La carpeta `radio` junto al resto del sitio.

4. Verificación en vivo

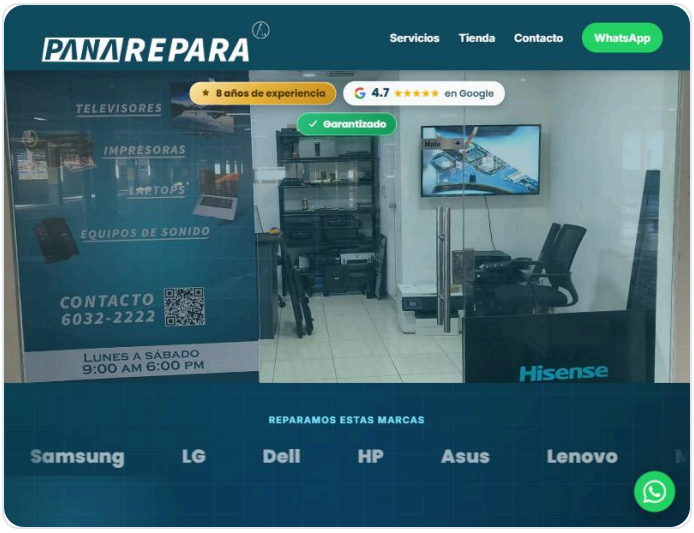
Se abrió la página publicada en un navegador real:



panarepara.com/radio/ en línea y **“Conectado”** al servidor MQTT. Marca “Sin datos” porque aún no hay ninguna placa encendida.



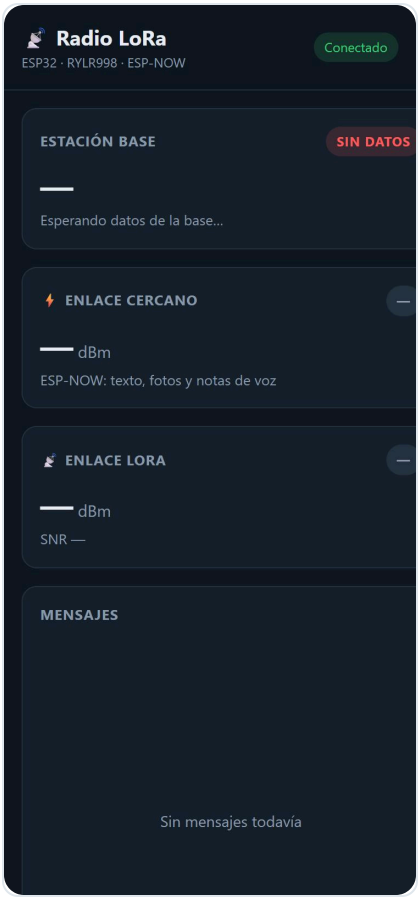
La misma página publicada en modo `?demo`.



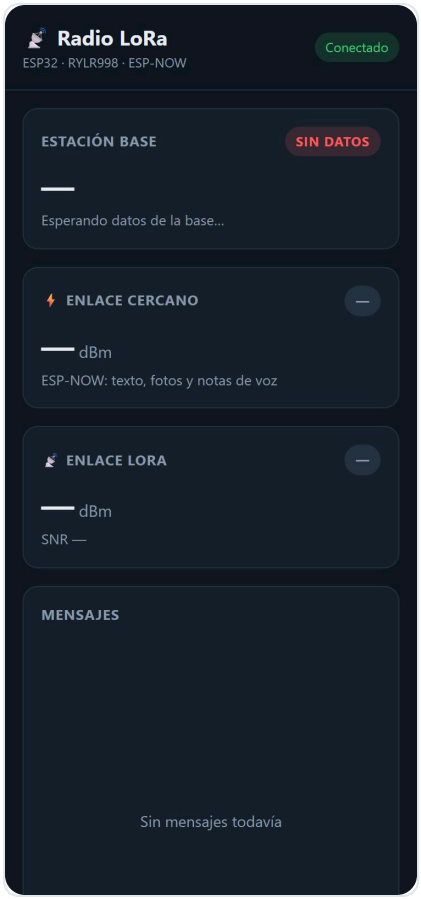
La portada de PanaRepara sigue igual: el cambio no afectó la tienda.

5. Problema encontrado y corregido

En la prueba con pantalla de celular (375 px) las tarjetas se cortaban por la derecha: la cuadrícula principal crecía al ancho mínimo de la barra de mensajes. Se corrigió con una línea de CSS (`grid-template-columns: minmax(0,1fr)`) y se publicó en el commit `6e91ba0` a las 23:33.



Antes: contenido cortado.



Después: todo cabe en la pantalla.

6. Estado actual

Etapa	Estado
Diseño del sistema	✔ Terminado
Firmware ESP32 (base y remota)	✔ Compila sin errores
Chat del celular	✔ Probado con datos simulados
Panel web en panarepara.com/radio/	✔ Publicado y conectado al servidor MQTT
Cargar el firmware en las dos placas	⏳ Pendiente
Confirmar que ambos módulos responden (“LoRa listo”)	⏳ Pendiente
Prueba de alcance ESP-NOW y LoRa	⏳ Pendiente